Projeto Debian Lab

Fase 1: Hardening do Debian

Relatório técnico da execução, com evidências

Documenta o que foi feito, por que foi feito e como cada resultado foi comprovado na VM `debian-lab`, do snapshot inicial ao estado final `hardening-v1`.

Autor: João Victor Paixão Zolim (chakal)

Versão: 1.0, 02/11/2026

Sistema-alvo: Debian 13 "Trixie" (VM KVM)

Resultado: Hardening index de 63 para 84

Versão pública: painel do desktop e nome do notebook borrados nos prints

Sumário

1. Resumo executivo . . . 3

2. Ambiente e método . . . 4

Ambiente do lab . . . 4

Regras de trabalho . . . 4

Snapshots criados . . . 5

3. Execução passo a passo . . . 5

1.0 Preparação . . . 5

1.1 Linha de base . . . 7

1.2 Atualizações automáticas de segurança . . . 8

1.3 Contas, senhas e sudo . . . 11

1.4 SSH só por chave . . . 14

1.5 Firewall com nftables . . . 16

1.6 fail2ban . . . 20

1.7 Montagens e partições . . . 26

1.8 Parâmetros do kernel . . . 31

1.9 Serviços, pacotes e SUID . . . 32

1.10 Auditoria com auditd . . . 34

1.11 AppArmor e AIDE . . . 35

1.12 Banner legal . . . 38

1.13 Auditoria final . . . 39

4. Problemas e aprendizados . . . 42

5. Desvios do roteiro . . . 42

6. Decisões sobre as sugestões restantes . . . 43

7. Limitações e pontos sem comprovação . . . 44

8. Próximos passos . . . 44

Apêndice A. Configurações finais . . . 45

1. Resumo executivo

Este relatório documenta a execução completa da **Fase 1 do Projeto Debian Lab** em 02/11/2026. Partimos de uma VM com Debian 13 em instalação mínima e chegamos a um servidor endurecido, com acesso administrativo só por chave, firewall em negação padrão, proteção contra força bruta, auditoria de eventos e verificação de integridade de arquivos.

O trabalho seguiu o ciclo **medir, corrigir e medir de novo**. Cada alteração foi testada de verdade: a Kali atacou a VM, arquivos sensíveis foram alterados para gerar alertas, e cada resultado ficou registrado em print ou arquivo.

HARDENING INDEX	AVISOS DO LYNIS	SUGESTÕES DO LYNIS	BINÁRIOS SUID
63 → **84**	2 → **1**	48 → **22**	11 → **6**
Lynis 3.1.4, mesma versão	o restante é esperado no lab	todas com decisão registrada	5 retirados com justificativa

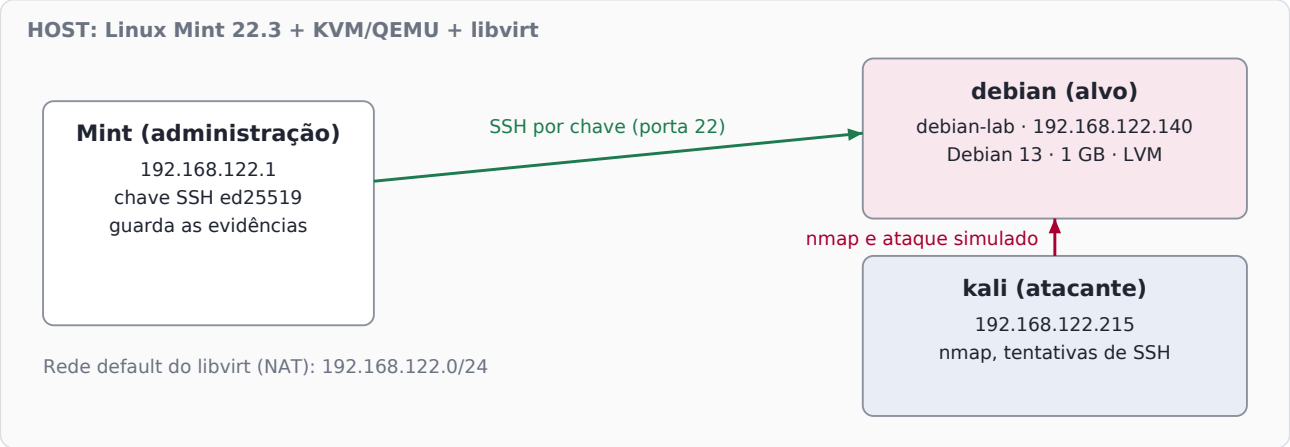
Área	Estado inicial	Estado final	Como foi comprovado
Atualizações	Pendentes, incluindo o kernel	Em dia, com instalação automática	`--dry-run` e timers do APT
Acesso remoto	SSH com senha	Só chave; root recusado; só `chakal`	3 testes de login em outro terminal
Firewall	Inexistente	`policy drop` em `input` e `forward`	`nmap -sS -p-` pela Kali: só a `22/tcp` aberta
Força bruta	Sem proteção	fail2ban: 5 falhas em 10 min, ban de 1 h	Kali banida e `Connection refused`
Montagens	`/tmp` e `/dev/shm` executáveis; logs na raiz	`noexec` nos dois; `/var/log` em LV próprio	Scripts bloqueados e reinício limpo
Kernel	16 parâmetros divergentes	15 corrigidos; 1 mantido de propósito	Nova auditoria do Lynis
Auditoria	Sem `auditd`	11 regras ativas	Evento com `auid=chakal` após `sudo`
Integridade	Sem verificação	Base AIDE do estado final	Detecção de um arquivo novo em `/etc`

O achado mais útil da fase
O AIDE revelou que a própria instalação dele trouxe o servidor de e-mail Exim como recomendação do pacote, abrindo a porta 25 em `localhost` e adicionando um binário SUID. O problema foi removido, e o `apt` passou a não instalar recomendações automaticamente. Os detalhes estão no passo 1.11.

Como ler a nota do Lynis
O hardening index é um indicador, não uma certificação. A nota 84 não significa que o sistema seja seguro em 84%. O valor real deste trabalho está nos testes práticos e nas decisões registradas nas seções 6 e 7.

2. Ambiente e método

Ambiente do lab



Item	Valor
Host	Notebook com Linux Mint 22.3, 16 GB de RAM, KVM/QEMU, libvirt e virt-manager
VM alvo	`debian`, hostname `debian-lab`, Debian 13 Trixie, 1 GB de RAM, 2 vCPU, disco de 20 GB em LVM
VM atacante	`kali`, Kali Linux, usada para varredura e tentativas de login
Rede	NAT do libvirt, sub-rede `192.168.122.0/24`
Kernel	De `6.12.113+deb13` para `6.12.115+deb13.1`
Ferramenta de medição	Lynis `3.1.4-1`, do repositório do Debian

Regras de trabalho

- **Um passo por vez:** cada alteração foi validada antes de seguir para a próxima.
- **Snapshot antes do risco:** SSH, firewall, PAM e partições só foram alterados depois de um ponto de retorno.
- **Sessão de segurança:** o console do virt-manager ou um shell de root ficaram abertos para evitar um bloqueio de acesso.
- **Drop-ins:** sempre que possível, as configurações ficaram em arquivos próprios, como `sshd_config.d/`, em vez de alterar os arquivos principais.
- **Teste a partir de fora:** o resultado só contou depois de ser confirmado por outra sessão ou pela Kali.

Snapshots criados

Os snapshots `running` guardam também a memória da VM, pois foram tirados com ela ligada. Os dois `shutoff` foram tirados com a VM desligada.

Hora	Snapshot	Estado	Finalidade
13:27:53	`pre-hardening`	shutoff	Antes da fase, ponto de retorno mais antigo
13:59:31	`pre-contas`	running	Antes de PAM e sudo
15:30:04	`pre-ssh`	running	Antes de desligar a senha no SSH
15:36:48	`pre-firewall`	running	Antes do nftables
16:51:14	`pre-fail2ban`	running	Antes do fail2ban
21:00:06	`pre-montagens`	running	Antes do `noexec`
21:09:34	`pre-varlog`	running	Antes de mover o `/var/log`
22:05:47	`hardening-v1`	shutoff	Estado final da Fase 1

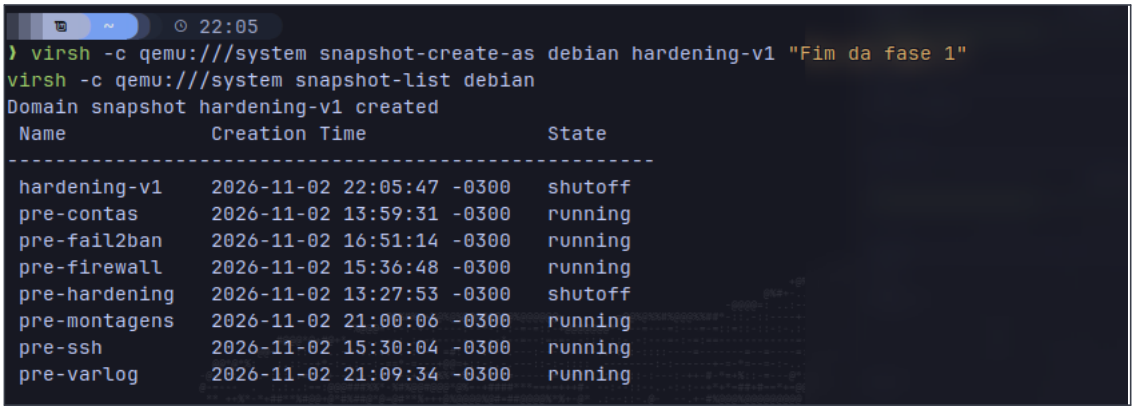


Figura 1. *Lista final com os 8 snapshots da fase.*

3. Execução passo a passo

Cada passo traz o objetivo, os comandos principais, as evidências e o resultado. Erros e correções foram mantidos, porque fazem parte do processo real. A seção 4 reúne esses problemas em um único lugar.

1.0 Preparação

Objetivo

Criar um ponto de retorno e passar a administrar a VM por SSH, em vez de usar o console.

O que foi feito

A VM já estava desligada. Criamos o snapshot, ligamos a máquina, descobrimos o IP pelo DHCP do libvirt e entramos por SSH.

```
# No Mint
virsh -c qemu:///system list --all
virsh -c qemu:///system snapshot-create-as debian pre-hardening "Antes da fase 1"
virsh -c qemu:///system start debian
virsh -c qemu:///system net-dhcp-leases default
ssh chakal@192.168.122.140

# No Debian
hostname; whoami; ip -4 -brief addr show
```

```
error: failed to connect to the hypervisor
error: internal error: invalid URI qemu://system (maybe you want qemu:///system)

 ~  13:26
❯ virsh -c qemu:///system list --all
 Id   Name     State
------------------------
 -    debian   shut off
 -    kali     shut off
 -    rocky    shut off
 -    ubuntu   shut off

 ~  13:26
❯
```

Figura 2. *Estado das VMs no início. O erro do topo foi um* `qemu://system` *digitado com duas barras. O certo é* `qemu:///system`*.*

```
 ~  13:26
❯ virsh -c qemu:///system snapshot-create-as debian pre-hardening "Antes da fase 1"
Domain snapshot pre-hardening created

 ~  13:27
❯ virsh -c qemu:///system snapshot-list debian
 Name            Creation Time               State
-----------------------------------------------------
 pre-hardening   2026-11-02 13:27:53 -0300   shutoff

 ~  13:28
❯
```

Figura 3. *Snapshot* `pre-hardening` *criado. Aqui notamos que o snapshot* `limpa`*, citado no roteiro, não existia.*

```
 ~  13:29
❯ virsh -c qemu:///system net-dhcp-leases default
 Expiry Time          MAC address         Protocol   IP address           Hostname     Client ID or DUI
D
-------------------------------------------------------------------------------------------------------------
-----------------------------------------
 2026-11-02 14:29:31   52:54:00:5b:7f:d5   ipv4       192.168.122.140/24   debian-lab   ff:00:5b:7f:d5:0
0:01:00:01:32:79:e6:85:52:54:00:5b:7f:d5

 ~  13:29
❯
```

Figura 4. *Concessão do DHCP com o IP* `192.168.122.140` *e o hostname* `debian-lab`*.*

```
chakal@debian-lab:~$ hostname; whoami; ip -4 -brief addr show
debian-lab
chakal
lo               UNKNOWN        127.0.0.1/8
enp1s0           UP             192.168.122.140/24
chakal@debian-lab:~$
```

Figura 5. *Primeiro acesso por SSH: máquina, usuário e IP confirmados.*

Resultado

A VM ficou acessível por SSH. O teclado passou a ser o do Mint, o que evitou digitar comandos no console.

> **Resumo para o README**
> **1.0 Preparação:** criei o snapshot `pre-hardening` antes de qualquer alteração e passei a administrar a VM por SSH a partir do host.

1.1 Linha de base

Objetivo

Medir o estado inicial sem alterar configurações, para que a comparação final tivesse valor. O Lynis audita o sistema e dá uma nota de 0 a 100, o hardening index.

O que foi feito

```
sudo apt update && sudo apt install -y lynis
apt policy lynis
sudo lynis audit system --no-colors | tee ~/lynis-antes.txt
sudo grep hardening_index /var/log/lynis-report.dat
sudo grep -E '^(warning|suggestion)\[\]=' /var/log/lynis-report.dat | tee ~/lynis-achados-antes.txt

# Inventário
sudo ss -tulpn | tee ~/portas-antes.txt
systemctl list-unit-files --state=enabled --no-pager | tee ~/servicos-antes.txt
sudo find / -xdev -perm -4000 -type f 2>/dev/null | sort | tee ~/suid-antes.txt
```

A opção `--no-colors` deixou o arquivo limpo para o GitHub. O `sudo` no `ss` mostra o processo dono de cada porta. Binários **SUID** rodam com os privilégios do dono do arquivo, por isso cada um é um possível caminho de escalada.

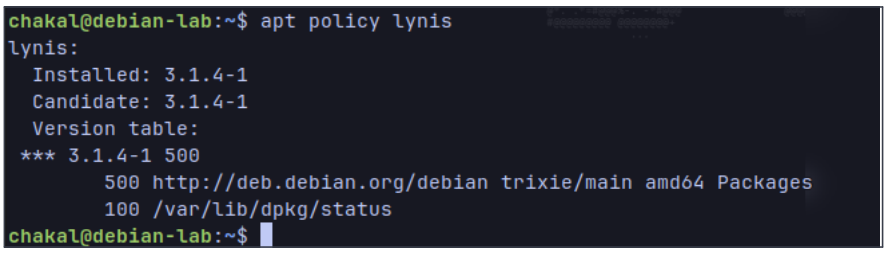


Figura 6. Lynis `3.1.4-1` do repositório do Debian. Esta versão foi mantida até o fim, para a comparação ser justa.

Resultado

Item	Resultado
Hardening index	**63**
Achados	2 avisos e 48 sugestões
Portas em escuta	TCP 22 (SSH), UDP 68 e 546 (cliente DHCP)
Serviços ativados	27 unidades, sem nada estranho
Binários SUID	11

```
warning[]=PKGS-7392|Found one or more vulnerable packages.|-|-|
warning[]=NETW-2705|Couldn't find 2 responsive nameservers|-|-|

tcp  LISTEN  0.0.0.0:22   users:(("sshd",pid=793,fd=6))
tcp  LISTEN  [::]:22      users:(("sshd",pid=793,fd=7))
```

O aviso `PKGS-7392` era real e foi corrigido no passo seguinte. O `NETW-2705` é esperado neste lab, porque a VM usa um único DNS, o da rede NAT.

> **Resumo para o README**
> **1.1 Linha de base:** a auditoria inicial com o Lynis 3.1.4 deu hardening index 63, com 2 avisos e 48 sugestões. Também registrei portas abertas, serviços ativados no boot e binários SUID para comparar no fim da fase.

1.2 Atualizações automáticas de segurança

Objetivo

Aplicar o que estava pendente e garantir que correções de segurança continuem chegando sem intervenção manual.

O que foi feito

```
sudo apt update
apt list --upgradable | tee ~/pacotes-pendentes.txt
sudo apt upgrade -y
sudo apt install -y unattended-upgrades apt-listchanges
sudo dpkg-reconfigure -plow unattended-upgrades
```

```
chakal@debian-lab:~$ sudo apt update \
> apt list --upgrade | tee ~/pacotes-pendentes.txt \
> sudo apt upgrade -y
tee: invalid option -- 'y'
Try 'tee --help' for more information.

WARNING: apt does not have a stable CLI interface. Use with caution in scripts.

Error: The update command takes no arguments
chakal@debian-lab:~$ sudo apt update
apt list --upgradable | tee ~/pacotes-pendentes.txt
sudo apt upgrade -y
Hit:1 http://deb.debian.org/debian trixie InRelease
Hit:2 http://deb.debian.org/debian trixie-updates InRelease
Hit:3 http://security.debian.org/debian-security trixie-security InRelease
3 packages can be upgraded. Run 'apt list --upgradable' to see them.

WARNING: apt does not have a stable CLI interface. Use with caution in scripts.

Listing...
linux-image-amd64/stable-security 6.12.115-2 amd64 [upgradable from: 6.12.113-1]
vim-common/stable-security 2:9.1.1230-2+deb13u1 all [upgradable from: 2:9.1.1230-2]
vim-tiny/stable-security 2:9.1.1230-2+deb13u1 amd64 [upgradable from: 2:9.1.1230-2]
The following packages were automatically installed and are no longer required:
  espeak-ng-data  libflac14       libogg0      libpulse0    libvorbis0a
  libasyncns0     libmp3lame0     libopus0     libsndfile1  libvorbisenc2
  libespeak-ng1   libmpg123-0t64  libpcaudio0  libsonic0    libx11-xcb1
Use 'sudo apt autoremove' to remove them.

Upgrading:
  linux-image-amd64  vim-common  vim-tiny

Installing dependencies:
  linux-image-6.12.115+deb13.1-amd64
```

Figura 7. *Três pacotes pendentes, incluindo o kernel. O erro no topo veio de barras \ no fim das linhas, que juntaram tudo em um só comando.*

```
Processing triggers for man-db (2.13.1-1) ...
chakal@debian-lab:~$ uname -r
6.12.113+deb13-amd64
chakal@debian-lab:~$ ls /boot/vmlinuz-*
/boot/vmlinuz-6.12.107+deb13-amd64  /boot/vmlinuz-6.12.115+deb13.1-amd64
/boot/vmlinuz-6.12.113+deb13-amd64
chakal@debian-lab:~$
```

Figura 8. *O kernel novo estava instalado, mas a VM ainda rodava o* `6.12.113`*. Foi preciso reiniciar.*

```
chakal@debian-lab:~$ cat /etc/apt/apt.conf.d/20auto-upgrades
cat: /etc/apt/apt.conf.d/20auto-upgrades: No such file or directory
chakal@debian-lab:~$ cat /etc/apt/apt.conf.d/20auto-upgrades
systemctl status unattended-upgrades --no-pager
systemctl list-timers 'apt-daily*' --no-pager
sudo unattended-upgrade --dry-run --debug 2>&1 | tail -n 15
cat: /etc/apt/apt.conf.d/20auto-upgrades: No such file or directory
Unit unattended-upgrades.service could not be found.
NEXT                        LEFT LAST                     PASSED UNIT                 ACTIVATES
Mon 2026-11-02 15:0… 3h 16min Sun 2026-11-01 07:0…         - apt-daily.timer         apt-daily.service
Tue 2026-11-03 06:4…      18h Mon 2026-11-02 11:3… 10min ago apt-daily-upgrade.ti… apt-daily-upgrade.se…

2 timers listed.
Pass --all to see loaded but inactive timers, too.
sudo: unattended-upgrade: command not found
chakal@debian-lab:~$
```

Figura 9. *Primeira validação: o* `unattended-upgrades` *ainda não estava instalado, porque essa etapa não tinha rodado.*

```
chakal@debian-lab:~$ mkdir -p ~/debian-hardening-lab/evidencias
scp 'chakal@192.168.122.140:~/*-antes.txt' ~/debian-hardening-lab/evidencias/
The authenticity of host '192.168.122.140 (192.168.122.140)' can't be established.
ED25519 key fingerprint is SHA256:BnF88gtnNsK1e5u/GFTtuX2591A3EOWDzJA8jYiLuas.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.122.140' (ED25519) to the list of known hosts.
chakal@192.168.122.140's password:
lynis-achados-antes.txt                                     100% 5190    12.1MB/s   00:00
lynis-antes.txt                                             100%   34KB  53.3MB/s   00:00
portas-antes.txt                                            100%  585     3.1MB/s   00:00
servicos-antes.txt                                          100% 1226     7.9MB/s   00:00
suid-antes.txt                                              100%  207     1.5MB/s   00:00
chakal@debian-lab:~$
```

Figura 10. *Erro de contexto: o* `scp` *rodou dentro do Debian, e a VM copiou os arquivos para ela mesma. O prompt* `chakal@debian-lab` *mostra isso.*

Corrigimos essa sequência em três etapas: apagamos a cópia indevida, reiniciamos a VM e refizemos a cópia a partir do Mint.

```
chakal@debian-lab:~$ rm -r ~/debian-hardening-lab
ssh-keygen -R 192.168.122.140
sudo reboot
# Host 192.168.122.140 found: line 1
# Host 192.168.122.140 found: line 2
# Host 192.168.122.140 found: line 3
/home/chakal/.ssh/known_hosts updated.
Original contents retained as /home/chakal/.ssh/known_hosts.old
chakal@debian-lab:~$ Connection to 192.168.122.140 closed by remote host.
Connection to 192.168.122.140 closed.
```

Figura 11. *Limpeza da cópia indevida e reinício para carregar o kernel novo.*

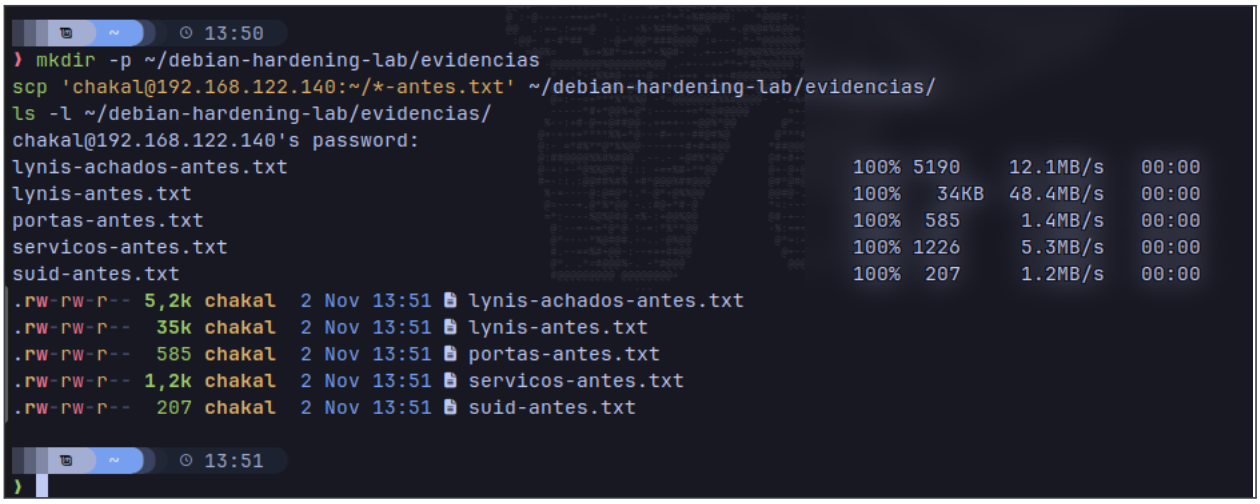


Figura 12. *Cópia correta: os 5 arquivos da linha de base chegaram ao Mint.*

Resultado

```
$ uname -r
6.12.115+deb13.1-amd64

$ cat /etc/apt/apt.conf.d/20auto-upgrades
APT::Periodic::Update-Package-Lists "1";
APT::Periodic::Unattended-Upgrade "1";

$ sudo unattended-upgrade --dry-run --debug 2>&1 | tail -n 15
No packages found that can be upgraded unattended and no pending auto-removals
upgrade result: True No packages found that can be upgraded unattended and no pending auto-removals
```

As duas opções estão em `"1"`, e a simulação terminou sem erros. O `unattended-upgrades.service` só cuida de atualizações em andamento no desligamento. Quem dispara a instalação diária é o `apt-daily-upgrade.timer`.

> **Limite conhecido**
> A VM não reinicia sozinha. Um kernel novo instalado automaticamente só entra em uso no próximo reinício manual.

Ajustes extras

O fuso da VM estava em EST, o que bagunçaria a linha do tempo da fase 3. Os avisos de locale vinham do SSH, que enviava configurações `pt_BR` inexistentes na VM. Os dois pontos foram corrigidos.

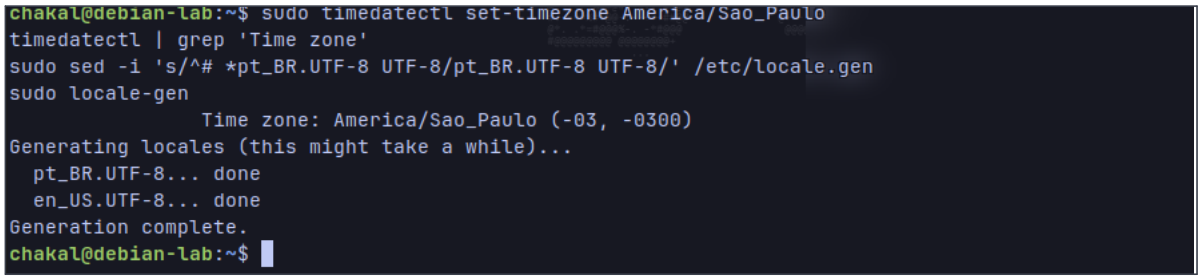


Figura 13. *Fuso ajustado para `America/Sao_Paulo` e locale `pt_BR.UTF-8` gerado.*

> **Resumo para o README**
> **1.2 Atualizações automáticas:** apliquei as atualizações pendentes, incluindo o kernel 6.12.115, e ativei o `unattended-upgrades`. A simulação com `--dry-run` rodou sem erros, e a instalação automática fica a cargo do `apt-daily-upgrade.timer`.

1.3 Contas, senhas e sudo

Objetivo

Reduzir o risco de senhas fracas e registrar o uso de privilégios. Como erros em PAM e sudo podem bloquear a administração, criamos o snapshot `pre-contas` e deixamos um shell de root aberto em outro terminal.



Figura 14. *Sessão de emergência com `sudo -i`. Ela continuaria funcionando mesmo se o sudo quebrasse.*



Figura 15. *Snapshot `pre-contas` criado antes de alterar o PAM e o sudo.*

Root e política de senha

```
sudo passwd -l root
sudo passwd -S root
sudo apt install -y libpam-pwquality libpwquality-tools

# /etc/security/pwquality.conf.d/hardening.conf
minlen = 14
minclass = 3
retry = 3
```

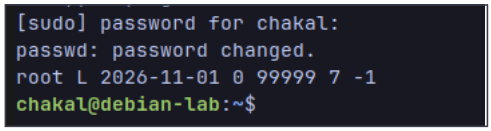


Figura 16. *O `L` confirma que a senha do root foi bloqueada. O `sudo` continua funcionando com a senha do `chakal`.*

A política foi testada com o `pwscore`, que avalia uma senha sem alterá-la. O pacote de teste foi removido depois.

```
$ echo 'Kp7mQz2xWd' | pwscore
Password quality check failed:
 The password is shorter than 14 characters

$ echo 'Kp7mQz2xWdLt9Rv4' | pwscore
56
```

```
chakal@debian-lab:~$ sudo apt purge -y libpwquality-tools
The following packages were automatically installed and are no longer required:
  espeak-ng-data  libmp3lame0     libpcaudio0  libvorbis0a
  libasyncns0     libmpg123-0t64  libpulse0    libvorbisenc2
  libespeak-ng1   libogg0         libsndfile1  libx11-xcb1
  libflac14       libopus0        libsonic0    linux-image-6.12.107+deb13-amd64
Use 'sudo apt autoremove' to remove them.

REMOVING:
  libpwquality-tools*

Summary:
  Upgrading: 0, Installing: 0, Removing: 1, Not Upgrading: 0
  Freed space: 50,2 kB

(Reading database ... 42781 files and directories currently installed.)
Removing libpwquality-tools (1.4.5-5) ...
Processing triggers for man-db (2.13.1-1) ...
chakal@debian-lab:~$
```

Figura 17. *Remoção do* `libpwquality-tools` *depois do teste, para não deixar ferramentas sem uso.*

Registro do sudo

```
# /etc/sudoers.d/hardening (editado com visudo)
Defaults use_pty
Defaults logfile="/var/log/sudo.log"
```

`use_pty` executa os comandos em um pseudo-terminal próprio, e `logfile` grava cada uso do sudo. O `visudo` validou a sintaxe, mas o arquivo nasceu com a permissão errada, o que corrigimos com `chmod 0440`.

```
chakal@debian-lab:~$ sudo visudo -f /etc/sudoers.d/hardening
chakal@debian-lab:~$ sudo visudo -c
/etc/sudoers: parsed OK
/etc/sudoers.d/README: parsed OK
/etc/sudoers.d/hardening: bad permissions, should be mode 0440
chakal@debian-lab:~$ sudo whoami
root
chakal@debian-lab:~$ sudo tail -n 3 /var/log/sudo.log
    COMMAND=/usr/bin/whoami
Nov  2 14:05:32 : chakal : TTY=pts/2 ; PWD=/home/chakal ; USER=root ;
    COMMAND=/usr/bin/tail -n 3 /var/log/sudo.log
chakal@debian-lab:~$
```

Figura 18. *O* `visudo -c` *acusou* `bad permissions`*, mas o log do sudo já funcionava.*

```
chakal@debian-lab:~$ sudo chown root:root /etc/sudoers.d/hardening
sudo chmod 0440 /etc/sudoers.d/hardening
sudo visudo -c
/etc/sudoers: parsed OK
/etc/sudoers.d/README: parsed OK
/etc/sudoers.d/hardening: parsed OK
chakal@debian-lab:~$ grep pwquality /etc/pam.d/common-password
password        requisite                       pam_pwquality.so retry=3
chakal@debian-lab:~$ sudo cp /etc/login.defs /etc/login.defs.orig
sudo sed -i -E 's/^PASS_MAX_DAYS.*/PASS_MAX_DAYS\t365/; s/^PASS_MIN_DAYS.*/PASS_MIN_DAYS\t1/' /etc/login
.defs
echo 'UMASK 027' | sudo tee -a /etc/login.defs
grep -E '^(PASS_MAX_DAYS|PASS_MIN_DAYS|PASS_WARN_AGE|UMASK)' /etc/login.defs
UMASK 027
PASS_MAX_DAYS   365
PASS_MIN_DAYS   1
PASS_WARN_AGE   7
UMASK 027
chakal@debian-lab:~$
```

Figura 19. *Permissão corrigida, módulo* `pam_pwquality` *confirmado na pilha do PAM e* `login.defs` *ajustado. O* `UMASK 027` *aparece duas vezes só porque o* `tee` *repete na tela o que grava.*

Validade de senhas e umask



Figura 20. *Revisão inicial: apenas* `root`*,* `sync` *e* `chakal` *têm shell válido, e a senha não expirava na prática (*`99999` *dias).*

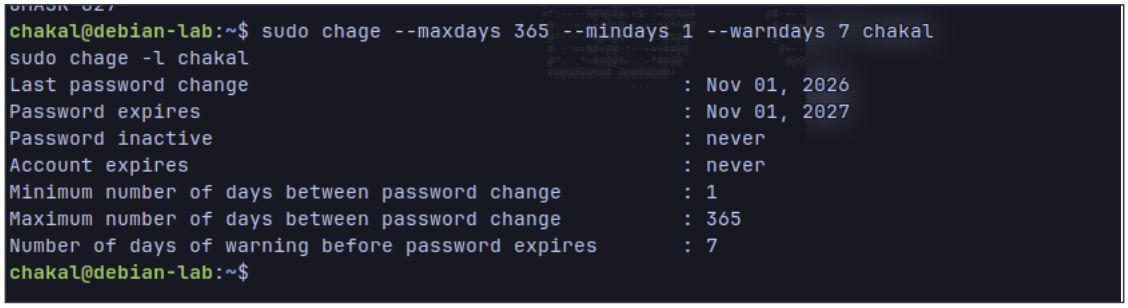


Figura 21. `chage` *aplicou as regras à conta existente, porque o* `login.defs` *só vale para contas novas.*

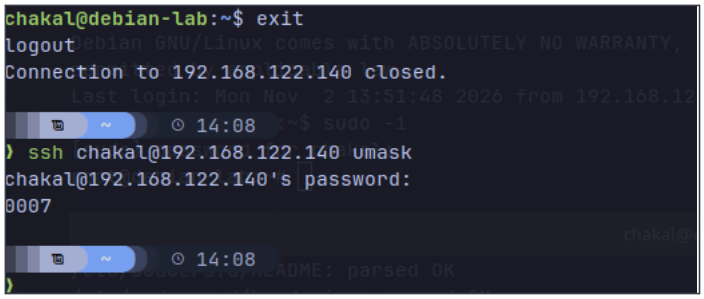


Figura 22. *O* `umask` *efetivo ficou em* `0007`*, não em* `0027`*.*

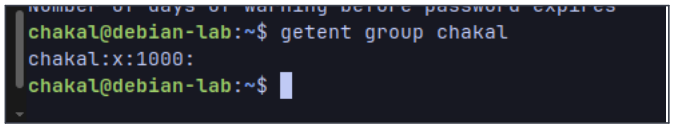


Figura 23. *O grupo* `chakal` *não tem outros membros, então o grupo não dá acesso a mais ninguém.*

O `pam_umask` iguala as permissões do grupo às do dono quando o usuário tem um grupo próprio. Assim, `027` vira `007`. Mantivemos esse valor porque outros usuários continuam sem acesso, e forçar `0027` exigiria alterar a pilha do PAM para um ganho pequeno.

> **Efeito colateral do UMASK**
> Arquivos criados com `sudo` passaram a nascer com permissões mais fechadas, como o modo `640`. Isso explica o `Permission denied` ao ler o `jail.local` no passo 1.6 e pode afetar serviços da fase 2.

> **Resumo para o README**
> **1.3 Contas e sudo:** bloqueei a senha do root, exigi senhas com 14 caracteres e 3 classes via `pwquality`, ativei o log do sudo e defini validade de senha. O `UMASK 027` resulta em `0007` para usuários com grupo próprio, porque o `pam_umask` iguala as permissões do grupo às do dono.

1.4 SSH só por chave

Objetivo

Eliminar ataques de senha contra o SSH e reduzir o que uma sessão pode fazer. Este era um passo arriscado, então seguimos esta ordem: criar snapshot, abrir o console, testar a chave e só então desligar a senha.



Figura 24. *Console do virt-manager aberto como rota de recuperação.*

Instalar a chave

A chave ed25519 já existia no Mint, então não geramos outra. Sobrescrever uma chave existente poderia cortar acessos atuais.

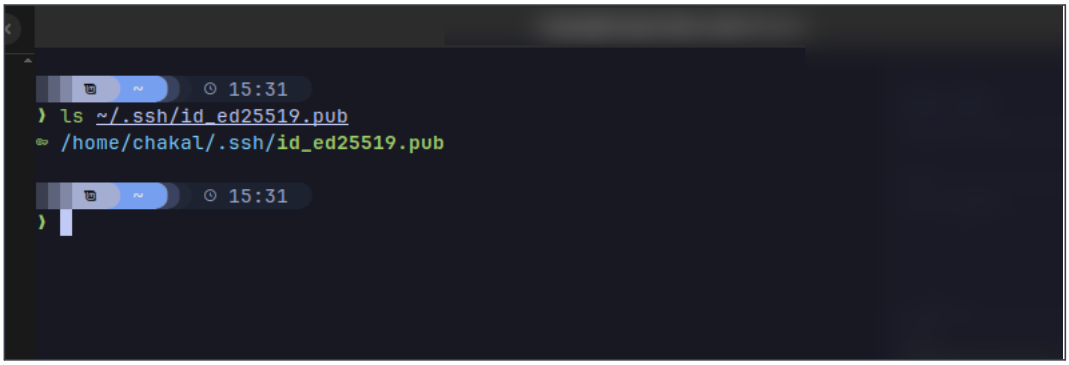


Figura 25. *Verificação de que a chave já existia.*



Figura 26. *Chave pública instalada. O teste com* `PasswordAuthentication=no` *provou que a chave funcionava antes de desligar a senha.*

Configuração

```
# /etc/ssh/sshd_config.d/10-hardening.conf
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
PubkeyAuthentication yes
AllowUsers chakal
MaxAuthTries 3
LoginGraceTime 30
X11Forwarding no
AllowTcpForwarding no
ClientAliveInterval 300
ClientAliveCountMax 2
Banner /etc/issue.net
AllowAgentForwarding no
MaxSessions 2
TCPKeepAlive no
LogLevel VERBOSE
```

Grupo	O que faz
Acesso	Bloqueia root e senha, exige chave e só permite o usuário `chakal`
Tentativas	Limita a 3 autenticações por conexão e dá 30 segundos para concluir o login
Recursos	Desliga encaminhamentos de X11, TCP e agente, que o lab não usa
Sessão	Derruba conexões sem resposta do cliente em cerca de 10 minutos
Registro	`LogLevel VERBOSE` registra qual chave foi usada no login, útil na fase 3

O banner em `/etc/issue.net` foi criado antes, porque o arquivo padrão poderia revelar o sistema antes do login. A sessão atual não caiu com o `reload`.

```
sudo sshd -t && sudo systemctl reload ssh
```

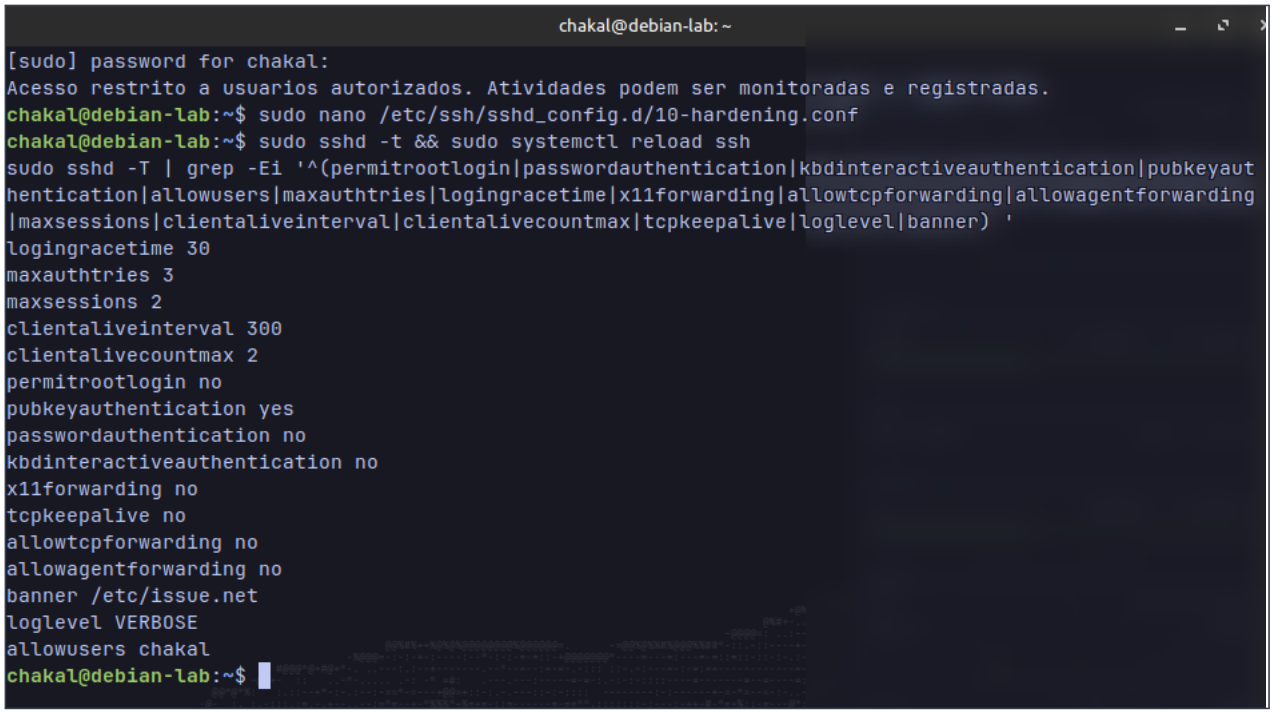


Figura 27. *O `sshd -T` mostra a configuração realmente em uso, e todos os valores conferem.*

Teste a partir de outro terminal

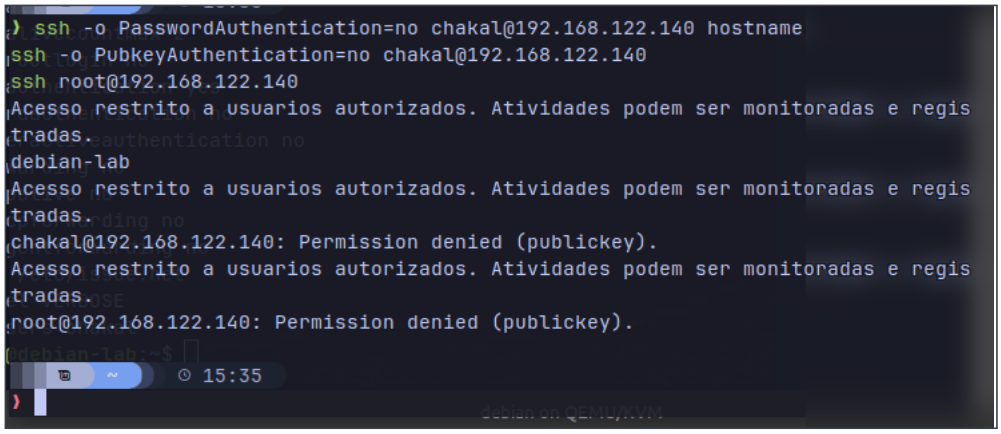


Figura 28. *Os três testes: a chave entrou, a tentativa sem chave foi recusada, e o root também.*

Teste	Esperado	Obtido
Login com chave	Banner e `debian-lab`	Passou
Login sem chave	`Permission denied (publickey)`	Passou
Login como root	`Permission denied (publickey)`	Passou

> **Resumo para o README**
> **1.4 SSH:** o acesso agora é só por chave ed25519, com root bloqueado e apenas o usuário `chakal` permitido. Usei um drop-in em `sshd_config.d`, reduzi tentativas e tempo de login, desliguei encaminhamentos e ativei `LogLevel VERBOSE`. Testei o login por chave e confirmei a recusa de senha e de root.

1.5 Firewall com nftables

Objetivo

Descartar por padrão tudo o que chega à VM e liberar apenas o necessário. O snapshot `pre-firewall` e o console ficaram prontos antes de aplicar as regras.

Regras

```
#!/usr/sbin/nft -f

flush ruleset

table inet filter {
    chain input {
        type filter hook input priority filter; policy drop;

        iif "lo" accept
        ct state established,related accept
        ct state invalid drop

        ip saddr 192.168.122.1 udp sport 67 udp dport 68 accept

        ip protocol icmp icmp type echo-request limit rate 5/second accept
        meta l4proto ipv6-icmp accept

        ip saddr 192.168.122.0/24 tcp dport 22 ct state new accept

        limit rate 5/minute log prefix "nft-drop: " counter drop
    }

    chain forward {
        type filter hook forward priority filter; policy drop;
```

```
    }

    chain output {
        type filter hook output priority filter; policy accept;
    }
}
```

- `policy drop`**:** tudo é descartado, a menos que uma regra libere.
- `lo` **e** `established,related`**:** liberam o tráfego interno e as respostas de conexões abertas. É isso que manteve a sessão SSH viva.
- **Porta 68:** regra que não estava no roteiro, adicionada para o DHCP do libvirt conseguir renovar o IP.
- **Porta 22:** novas conexões SSH só da rede do lab.
- **Última regra:** registra até 5 pacotes por minuto, evitando que uma varredura encha o disco.

```
sudo apt install -y nftables
sudo cp /etc/nftables.conf /etc/nftables.conf.orig
sudo nft -c -f /etc/nftables.conf     # só confere a sintaxe
sudo systemctl enable --now nftables
sudo nft list ruleset
```

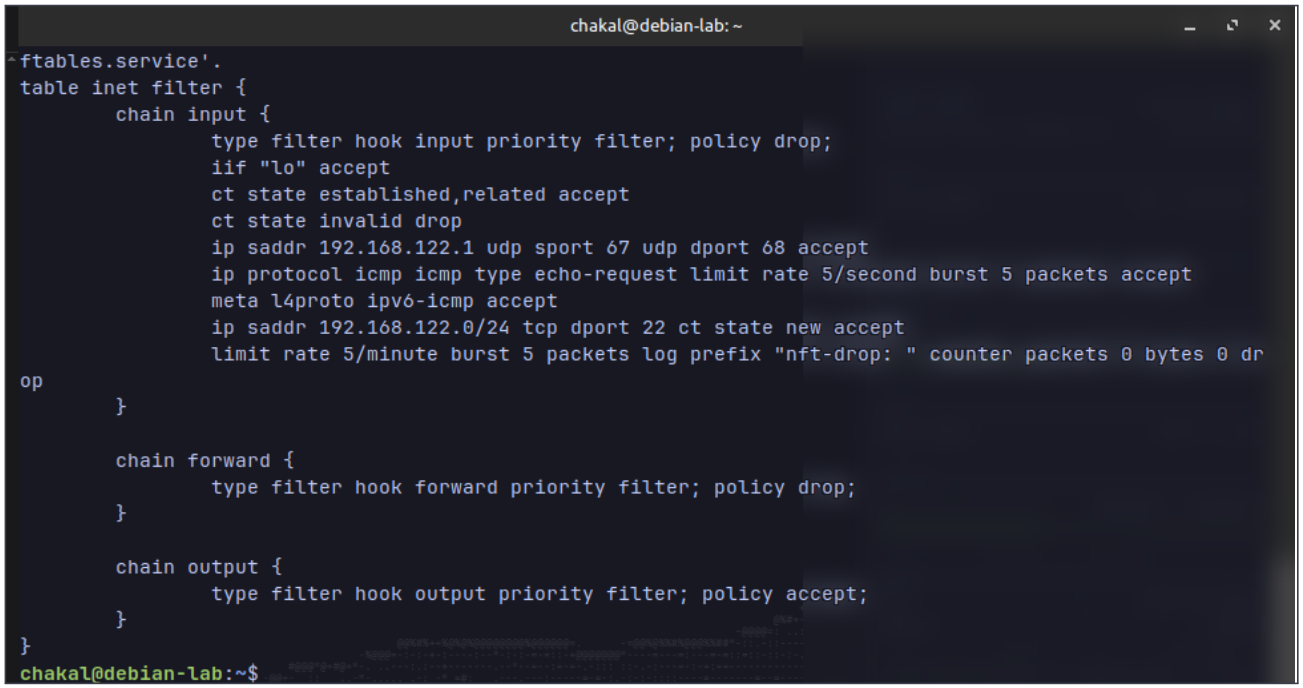


Figura 29. *Regras carregadas. A mensagem* `AF_VSOCK` *no console é só um aviso de que a VM não tem esse dispositivo.*

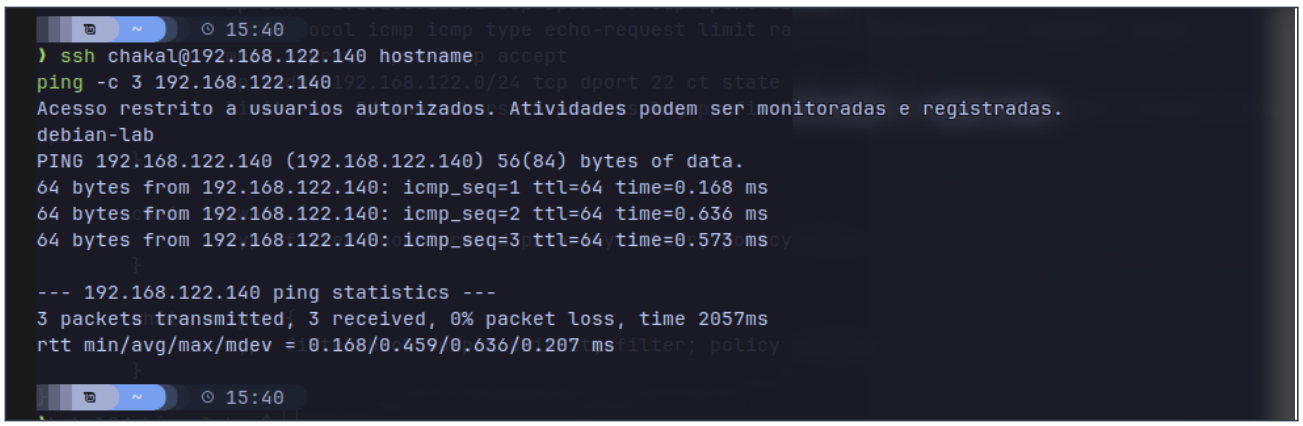


Figura 30. *Depois de aplicar as regras, um novo login por SSH funcionou e o ping respondeu.*

```
}
chakal@debian-lab:~$ sudo apt update
Hit:1 http://security.debian.org/debian-security trixie-security InRelease
Hit:2 http://deb.debian.org/debian trixie InRelease
Hit:3 http://deb.debian.org/debian trixie-updates InRelease
All packages are up to date.
chakal@debian-lab:~$
```

Figura 31. *A VM continuou acessando a internet, já que a saída segue permitida.*

Teste de ataque pela Kali

```
~ 15:42
❯ virsh -c qemu:///system start kali
virsh -c qemu:///system net-dhcp-leases default
Domain 'kali' started

 Expiry Time           MAC address         Protocol   IP address           Hostname     Client ID or DUI
D
-------------------------------------------------------------------------------------------------------
----------------------------------------
 2026-11-02 16:41:29   52:54:00:22:75:aa   ipv4       192.168.122.215/24   kali         01:52:54:00:22:7
5:aa
 2026-11-02 16:28:44   52:54:00:5b:7f:d5   ipv4       192.168.122.140/24   debian-lab   ff:00:5b:7f:d5:0
0:01:00:01:32:79:e6:85:52:54:00:5b:7f:d5


~ 15:42
❯
```

Figura 32. *Kali ligada, com o IP* `192.168.122.215`*.*

```
~ 15:42
❯ ssh 192.168.122.215
ssh: connect to host 192.168.122.215 port 22: Connection refused

~ 15:43
❯ ssh kali@192.168.122.215
The authenticity of host '192.168.122.215 (192.168.122.215)' can't be established.
ED25519 key fingerprint is SHA256:z+Ap3bwSO7x5M72s4p5rNogwyXNaReu4vOSrGJLnwc8.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.122.215' (ED25519) to the list of known hosts.
kali@192.168.122.215's password:
Linux kali 7.1.5+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 7.1.5-1kali1 (2026-07-29) x86_64

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
┌──(kali㉿kali)-[~]
└─$
```

Figura 33. *O primeiro* `ssh` *tentou entrar como* `chakal` *e encontrou o serviço desligado. Com o SSH ativo na Kali, entramos como* `kali`*.*

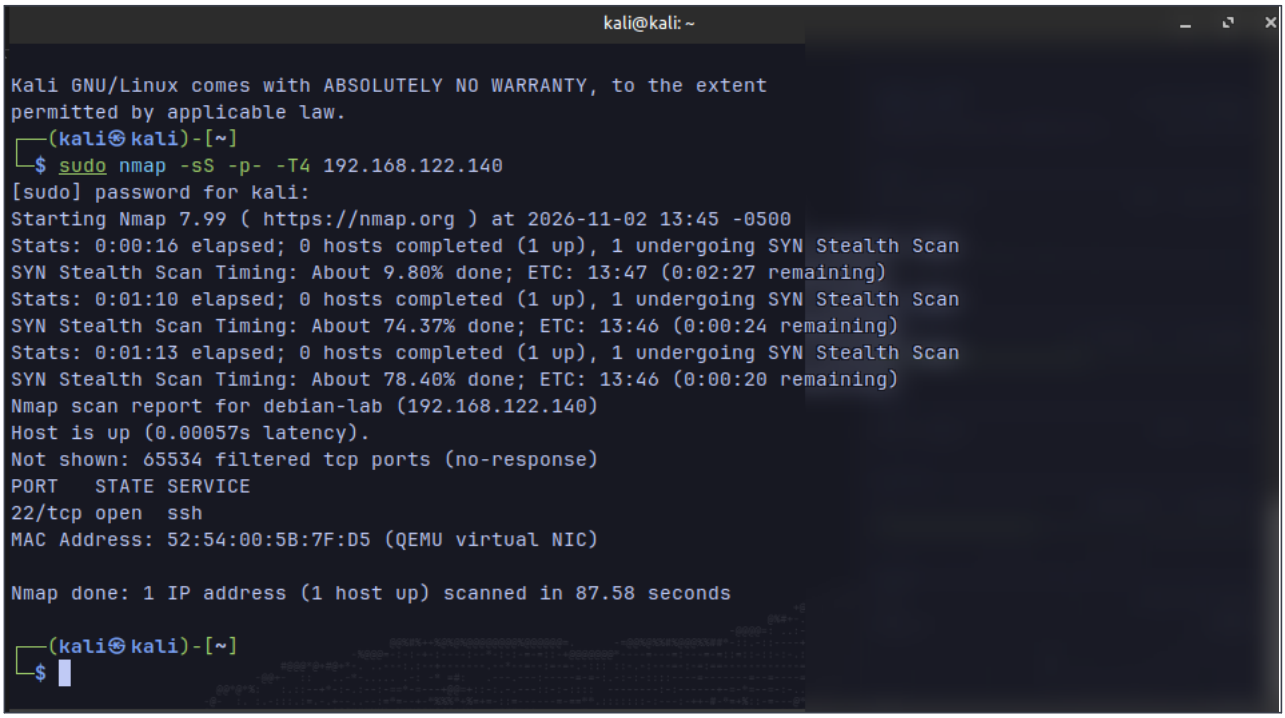


Figura 34. *Varredura de todas as portas TCP: só a `22/tcp` aberta, com 65.534 portas filtradas.*

Portas descartadas não respondem, e por isso aparecem como `filtered`. Isso também explica os 87 segundos da varredura, já que o `nmap` precisa esperar o tempo limite.

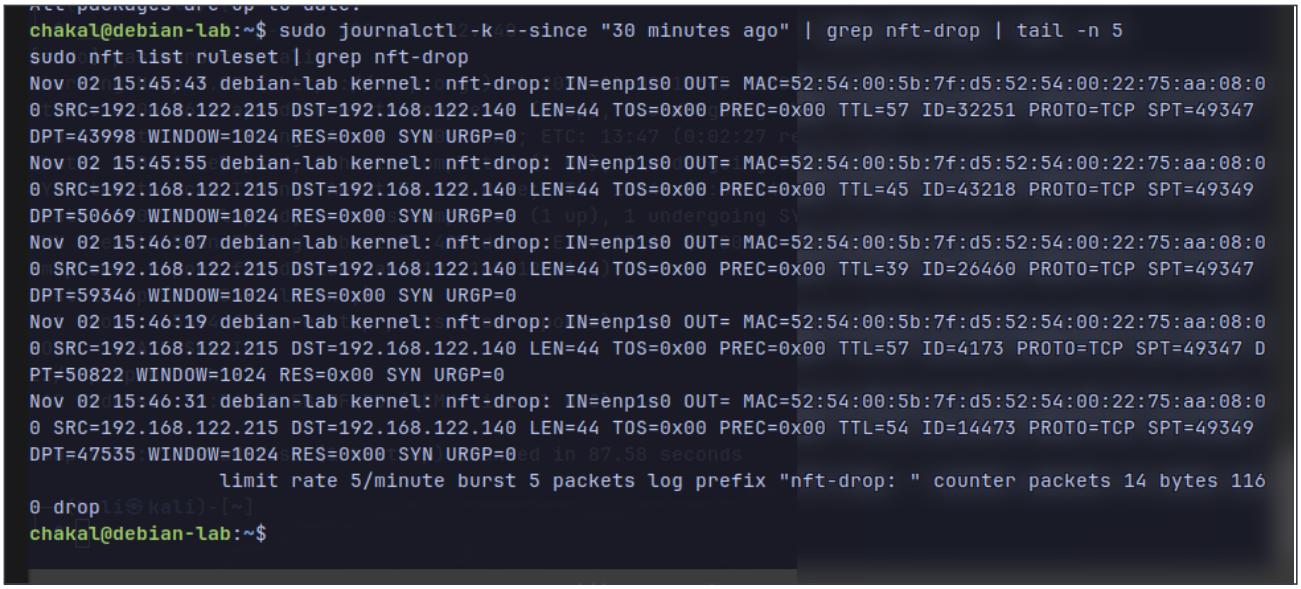


Figura 35. *Detecção: pacotes SYN da Kali para portas diferentes, um padrão típico de varredura. O intervalo de 12 segundos vem do limite de 5 registros por minuto.*

O contador ficou em 14 porque só conta os pacotes que passaram pelo limite da regra. Os demais foram descartados pela `policy drop`, sem gerar log. Portanto, ele **não representa** o total bloqueado.

Persistência



Figura 36. *Depois do reinício, os comandos seguintes foram colados junto com o* `ssh` *e não chegaram a rodar dentro do Debian.*

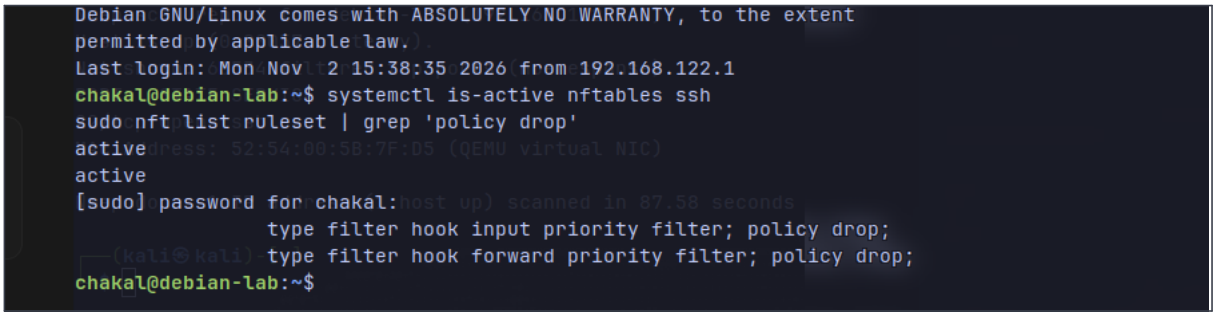


Figura 37. *Refeita dentro da VM, a checagem mostra serviços ativos e as duas* `policy drop` *carregadas.*

> **Teclado**
> Durante este passo, descobrimos que o teclado físico é US e o console estava em ABNT2. Alteramos `XKBLAYOUT` para `us` em `/etc/default/keyboard` e reiniciamos a VM. O teste confirmou que o console passou a funcionar normalmente.

> **Alcance do teste**
> A varredura cobriu **TCP sobre IPv4**. As regras liberam SSH apenas para o IPv4 do lab, mas não fizemos varredura UDP nem IPv6. Isso pode entrar como teste extra na fase 3.

> **Resumo para o README**
> **1.5 Firewall:** configurei o nftables com `policy drop` em `input` e `forward`, liberando SSH apenas da rede do lab e DHCP do gateway. A varredura `nmap -sS -p-` pela Kali encontrou só a porta 22 aberta, com 65.534 portas filtradas. Os bloqueios apareceram no log com o prefixo `nft-drop`, e as regras persistiram após reiniciar.

1.6 fail2ban

Objetivo

Bloquear automaticamente IPs com falhas repetidas no SSH. Como o Debian 13 não grava `/var/log/auth.log`, o fail2ban lê o journal do systemd.

Configuração

```
# /etc/fail2ban/jail.local
[DEFAULT]
backend = systemd
banaction = nftables-multiport
bantime = 1h
findtime = 10m
maxretry = 5
ignoreip = 127.0.0.1/8 ::1 192.168.122.1

[sshd]
enabled = true
mode = aggressive
```

- **5 falhas em 10 minutos** geram **1 hora** de ban.
- `mode = aggressive` também pega falhas com chave, que é o que sobra sem login por senha.
- `ignoreip` não estava no roteiro e impede que o Mint seja banido por engano. Por isso, o ataque precisava partir da Kali.

Dois problemas antes do teste



Figura 38. *O `cat` sem `sudo` recebeu `Permission denied`: o arquivo nasceu com modo `640`.*

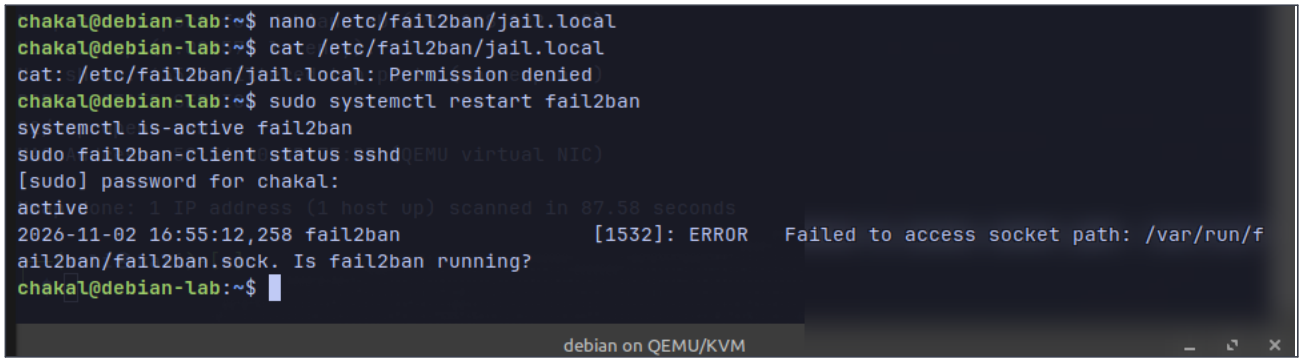


Figura 39. *O `Failed to access socket` veio da pressa: o `status` rodou enquanto o serviço ainda subia.*

```
$ sudo ls -l /etc/fail2ban/jail.local
-rw-r----- 1 root root 1 Nov  2 16:52 /etc/fail2ban/jail.local

$ sudo fail2ban-client -t
OK: configuration test is successful
```

O diagnóstico revelou o problema real: o `jail.local` tinha só **1 byte**, ou seja, o conteúdo não estava salvo. Além disso, o `nano` aberto sem `sudo` não conseguiria gravar nele. O fail2ban rodava com os padrões do Debian, sem as nossas regras, e o teste de sintaxe passava mesmo assim. Regravamos o arquivo com `sudo tee` e confirmamos os valores carregados:

```
$ sudo fail2ban-client get sshd maxretry
5
$ sudo fail2ban-client get sshd bantime
3600
$ sudo fail2ban-client get sshd ignoreip
These IP addresses/networks are ignored:
|- 127.0.0.0/8
|- 192.168.122.1
`- ::1
```

Ataque simulado

```
# Na Kali
for i in $(seq 1 8); do
  ssh -o BatchMode=yes -o ConnectTimeout=5 -o StrictHostKeyChecking=accept-new \
    teste$i@192.168.122.140
done
```

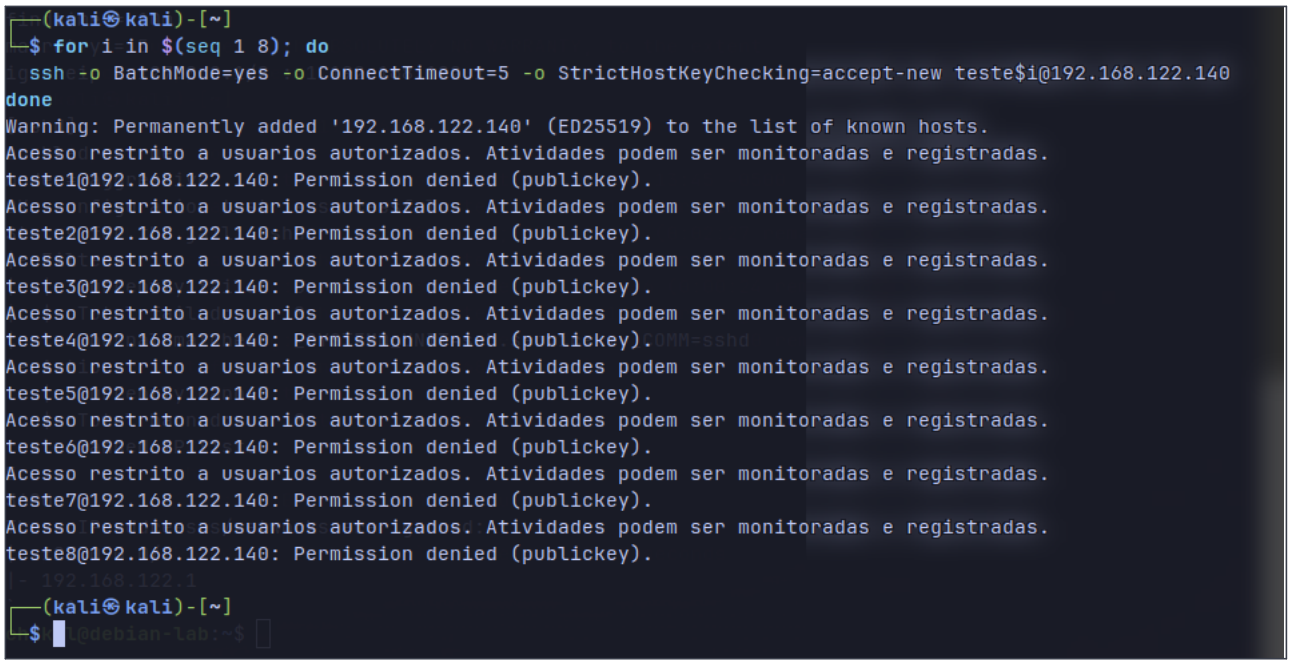


Figura 40. *As 8 tentativas com usuários inexistentes, todas recusadas pelo SSH.*

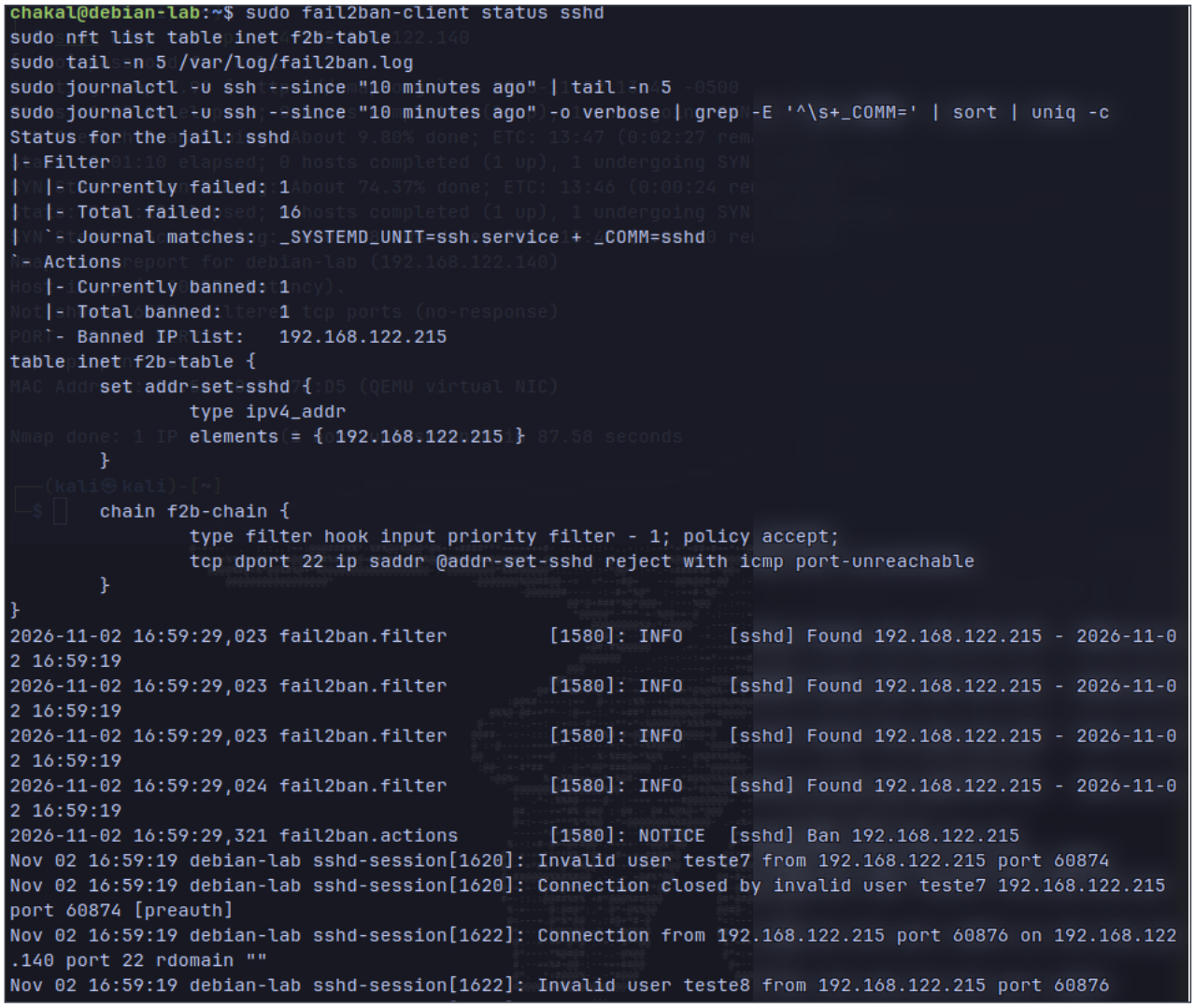


Figura 41. *Kali banida às 16:59:29, com 16 falhas contadas e o IP no conjunto* `addr-set-sshd`*.*

O ban veio cerca de 10 segundos depois das tentativas. Como o loop terminou em cerca de 1 segundo, ele bloqueou as conexões seguintes, não as que já tinham passado. A regra usa `reject`, por isso o atacante recebe uma recusa imediata em vez de esperar um tempo limite.

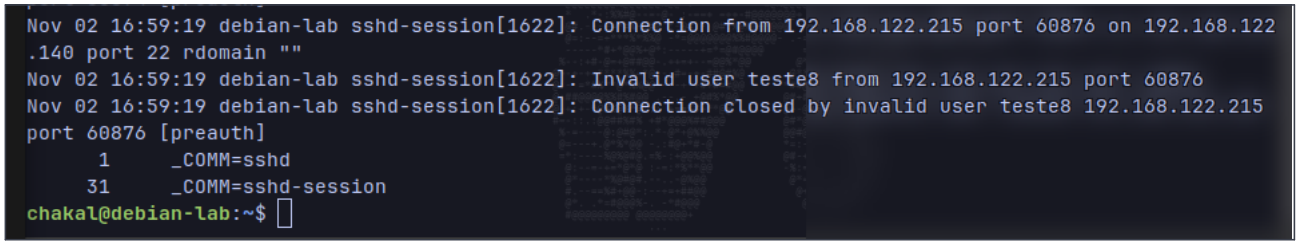


Figura 42. *31 das 32 entradas analisadas vieram de* `sshd-session`*, o processo que trata as sessões nas versões recentes do OpenSSH.*

O filtro exibe `_SYSTEMD_UNIT=ssh.service + _COMM=sshd`. No journal, o `+` funciona como **OU**, então a unidade `ssh.service` já cobre os logs do `sshd-session`. Por isso, nenhuma correção foi necessária.

Comprovar o bloqueio

O primeiro teste de bloqueio foi feito horas depois, e a Kali conseguiu entrar até o banner. A causa foi simples: o ban já tinha expirado. O log confirmou o `Unban` às 17:59:19, cerca de 1 hora depois das falhas. Refizemos o teste com um ban manual.

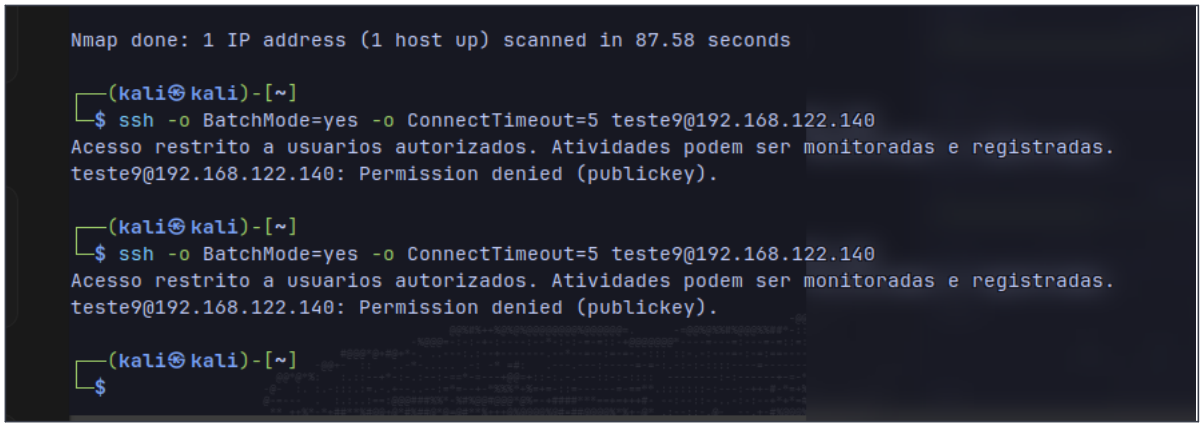


Figura 43. *Com o ban expirado, a Kali voltou a chegar ao SSH nas duas tentativas.*

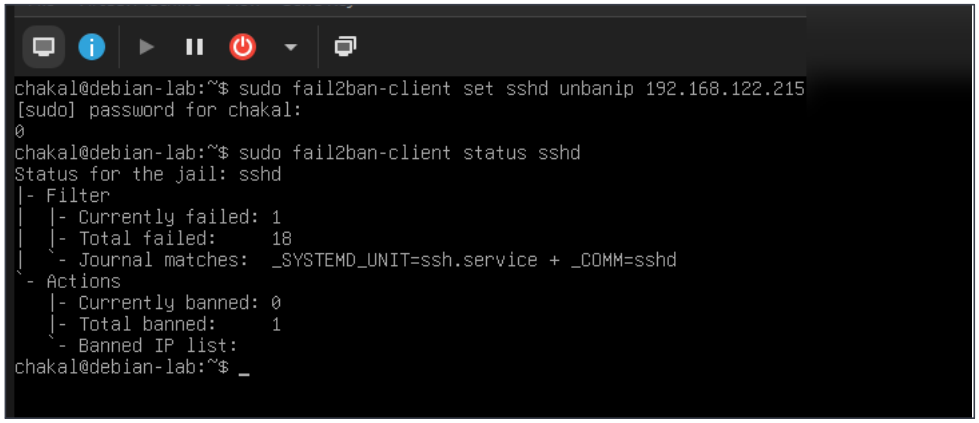


Figura 44. *O `unbanip` retornou `0`, sinal de que não havia ban ativo para remover.*



Figura 45. *O log mostra o fim do ban às 17:59:19. Em seguida, a Kali é banida manualmente.*

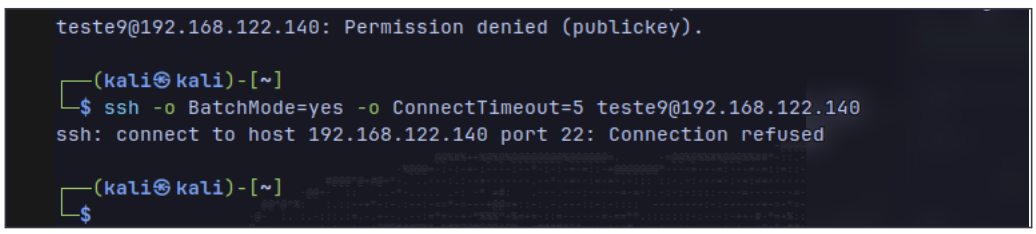


Figura 46. *Durante o ban, a conexão é recusada antes de chegar ao SSH.*

```
}
chakal@debian-lab:~$ sudo fail2ban-client set sshd unbanip 192.168.122.215
sudo fail2ban-client status sshd
1
Status for the jail: sshd
|- Filter
|  |- Currently failed: 1
|  |- Total failed:     20
|  `- Journal matches:  _SYSTEMD_UNIT=ssh.service + _COMM=sshd
`- Actions
   |- Currently banned: 0
   |- Total banned:     2
   `- Banned IP list:
chakal@debian-lab:~$
```

Figura 47. Liberação com `unbanip`, que retorna `1` quando havia um ban ativo.

```
    ssh: connect to host 192.168.122.140 port 22: Connection refused

    ┌──(kali㉿kali)-[~]
    └─$ ssh -o BatchMode=yes -o ConnectTimeout=5 teste9@192.168.122.140
    Acesso restrito a usuarios autorizados. Atividades podem ser monitoradas e registradas.
    teste9@192.168.122.140: Permission denied (publickey).

    ┌──(kali㉿kali)-[~]
    └─$
```

Figura 48. Depois da liberação, o banner volta a aparecer.

Um ban que voltou depois do reinício

Depois do passo 1.7, o log mostrou `Restore Ban` para a Kali. A última tentativa de teste tinha somado com as anteriores dentro da janela de 10 minutos e atingido o limite, gerando um ban automático às 20:56:42. O fail2ban guarda os bans e os restaura enquanto ainda estão no prazo. Esse é o comportamento certo, e bastou liberar a Kali de novo.

```
2026-11-02 21:12:38,914 fail2ban.actions        [797]: NOTICE  [sshd] Restore Ban 192.168.122.215
chakal@debian-lab:~$ sudo grep -E 'Ban |Unban ' /var/log/fail2ban.log | tail -n 6
sudo fail2ban-client set sshd unbanip 192.168.122.215
sudo fail2ban-client status sshd
2026-11-02 20:56:23,372 fail2ban.actions        [1580]: NOTICE  [sshd] Unban 192.168.122.215
2026-11-02 20:56:42,234 fail2ban.actions        [1580]: NOTICE  [sshd] Ban 192.168.122.215
2026-11-02 21:02:42,530 fail2ban.actions        [1580]: NOTICE  [sshd] Unban 192.168.122.215
2026-11-02 21:02:53,990 fail2ban.actions        [787]: NOTICE  [sshd] Restore Ban 192.168.122.215
2026-11-02 21:10:24,838 fail2ban.actions        [787]: NOTICE  [sshd] Unban 192.168.122.215
2026-11-02 21:12:38,914 fail2ban.actions        [797]: NOTICE  [sshd] Restore Ban 192.168.122.215
1
Status for the jail: sshd
|- Filter
|  |- Currently failed: 0
|  |- Total failed:     0
|  `- Journal matches:  _SYSTEMD_UNIT=ssh.service + _COMM=sshd
`- Actions
   |- Currently banned: 0
   |- Total banned:     1
   `- Banned IP list:
chakal@debian-lab:~$
```

Figura 49. Sequência completa de `Ban`, `Unban` e `Restore Ban`, e a Kali liberada no final.

> **Cuidado com o nftables**
> O `flush ruleset` do `nftables.conf` apaga também as regras do fail2ban. Sempre que recarregar o nftables, rode `sudo systemctl restart fail2ban` em seguida.

> **Resumo para o README**
> **1.6 fail2ban:** configurei a jail `sshd` com backend `systemd` e `mode = aggressive`: 5 falhas em 10 minutos geram 1 hora de ban via nftables. Em ataque simulado pela Kali, o IP foi banido cerca de 10 segundos após as tentativas. Durante o ban, novas conexões receberam `Connection refused`, e o log registrou o `Unban` 1 hora depois. O host administrativo fica no `ignoreip`.

1.7 Montagens e partições

Objetivo

Impedir a execução direta de programas em diretórios onde qualquer usuário escreve e separar os logs da raiz, para que logs lotados não encham o sistema.

Diagnóstico

```
TARGET SOURCE FSTYPE OPTIONS
/tmp   tmpfs  tmpfs  rw,nosuid,nodev,size=495144k,nr_inodes=1048576,inode64
TARGET   SOURCE FSTYPE OPTIONS
/dev/shm tmpfs  tmpfs  rw,nosuid,nodev,inode64
TARGET SOURCE                      FSTYPE OPTIONS
/      /dev/mapper/chakal--vg-root ext4   rw,relatime,errors=remount-ro
TARGET SOURCE                      FSTYPE OPTIONS
/      /dev/mapper/chakal--vg-root ext4   rw,relatime,errors=remount-ro
  VG        #PV #LV #SN Attr   VSize  VFree
  chakal-vg   1   2   0 wz--n- 19,05g <5,09g
  LV     VG        Attr       LSize   Pool Origin Data%  Meta%  Move Log Cpy%Sync Convert
  root   chakal-vg -wi-ao---- <13,20g
  swap_1 chakal-vg -wi-ao---- 788,00m
```

Figura 50. *Estado inicial: `/tmp` e `/dev/shm` já têm `nosuid` e `nodev`, mas não `noexec`. O VG tem 5,09 GB livres.*

```
[Mount]
What=tmpfs
Where=/tmp
Type=tmpfs
Options=mode=1777,strictatime,nosuid,nodev,size=50%%,nr_inodes=1m
33M     /var/log
drwxr-sr-x+ 3 root systemd-journal 4096 Nov  1 09:01 /var/log/journal
/dev/mapper/chakal--vg-root /               ext4    errors=remount-ro 0       1
UUID=0a02cf5d-bae2-45a7-bcc7-04be4ed0c11a /boot           ext4    defaults        0       2
/dev/mapper/chakal--vg-swap_1 none            swap    sw              0       0
/dev/sr0        /media/cdrom0   udf,iso9660 user,noauto     0       0
chakal@debian-lab:~$
```

Figura 51. *Opções do `/tmp`, tamanho do `/var/log` (33 MB) e `fstab` inicial.*

Dividimos o trabalho em duas partes: primeiro os diretórios em memória, que são mais simples, e depois o `/var/log`, que é mais delicado.

Parte A: `noexec` em `/tmp` e `/dev/shm`

```
# /etc/systemd/system/tmp.mount.d/hardening.conf
[Mount]
Options=mode=1777,strictatime,nosuid,nodev,noexec,size=50%%,nr_inodes=1m

# /etc/fstab
tmpfs /dev/shm tmpfs defaults,nodev,nosuid,noexec 0 0
```

O `%%` é a forma de escrever `%` em uma unit do systemd. O `findmnt --verify` conferiu o `fstab` antes do reinício, já que um erro nesse arquivo pode travar o boot.

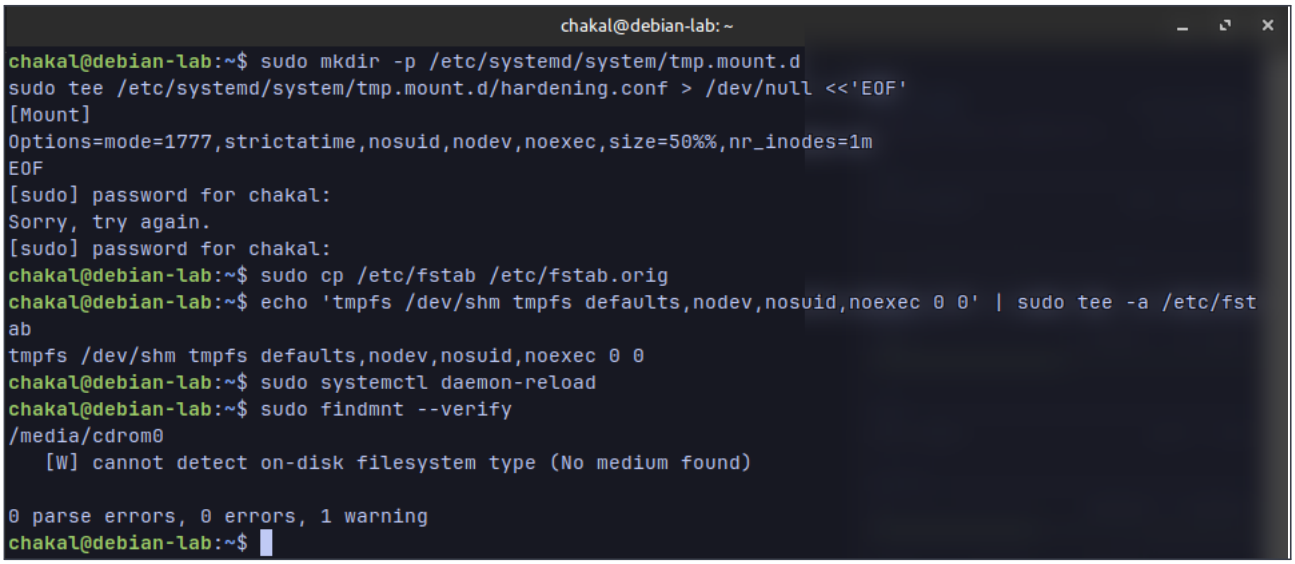


Figura 52. *Validação com* `0 errors`*. O único aviso é do CD-ROM vazio, e o* `Sorry, try again` *foi uma senha digitada errada.*

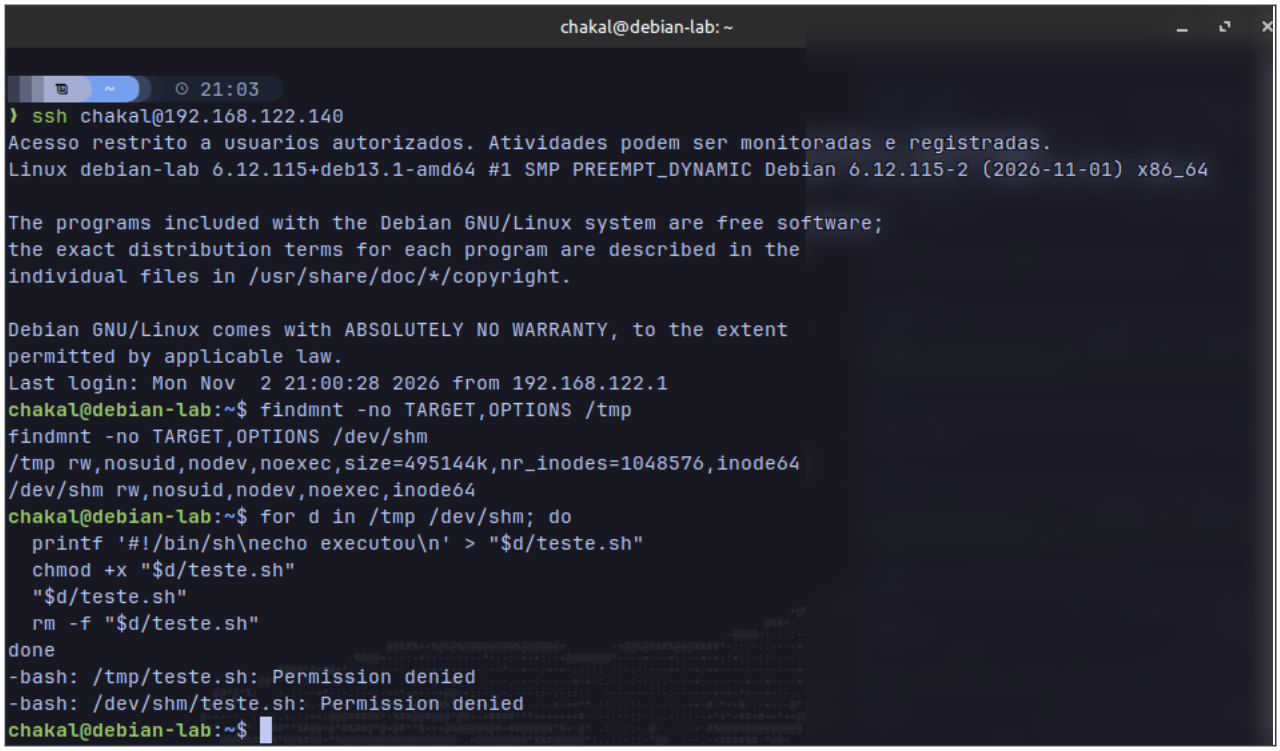


Figura 53. *Depois do reinício, as duas montagens têm* `noexec`*, e os scripts recebem* `Permission denied`*.*

O roteiro alertava que o `noexec` no `/tmp` poderia quebrar o `apt`. Reinstalamos um pacote que usa debconf para testar, e nenhum erro apareceu.

```
chakal@debian-lab:~$ sudo apt-get install --reinstall -y unattended-upgrades 2>&1 | tail -n 15
[sudo] password for chakal:
Sorry, try again.
[sudo] password for chakal:
  espeak-ng-data libasyncns0 libespeak-ng1 libflac14 libmp3lame0
  libmpg123-0t64 libogg0 libopus0 libpcaudio0 libpulse0 libsndfile1 libsonic0
  libvorbis0a libvorbisenc2 libx11-xcb1 linux-image-6.12.107+deb13-amd64
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 1 reinstalled, 0 to remove and 0 not upgraded.
Need to get 66,9 kB of archives.
After this operation, 0 B of additional disk space will be used.
Get:1 http://deb.debian.org/debian trixie/main amd64 unattended-upgrades all 2.12 [66,9 kB]
Preconfiguring packages ...
Fetched 66,9 kB in 0s (727 kB/s)
(Reading database ... 44013 files and directories currently installed.)
Preparing to unpack .../unattended-upgrades_2.12_all.deb ...
Unpacking unattended-upgrades (2.12) over (2.12) ...
Setting up unattended-upgrades (2.12) ...
Processing triggers for man-db (2.13.1-1) ...
chakal@debian-lab:~$
```

Figura 54. `Preconfiguring packages` *passou sem* `Can't exec` *nem* `Permission denied`.

> **O que o noexec não faz**
> Ele é uma camada de defesa, não uma barreira completa. Por exemplo, `sh /tmp/teste.sh` ainda rodaria, porque quem executa é o `/bin/sh`.

Parte B: `/var/log` em volume próprio

Aqui mudamos o método do roteiro. O `rescue.target` depende da senha do root para o console, e essa senha estava bloqueada desde o passo 1.3. Por isso, copiamos os logs com o sistema no ar e reiniciamos. Na volta, o systemd monta o volume novo antes de os serviços gravarem em disco. O custo é perder os logs dos poucos minutos entre a cópia e o reinício, o que é aceitável no lab.

```
sudo lvcreate -L 2G -n varlog chakal-vg
sudo mkfs.ext4 /dev/chakal-vg/varlog
sudo mkdir /mnt/newlog
sudo mount /dev/chakal-vg/varlog /mnt/newlog
sudo systemctl stop fail2ban
sudo cp -a /var/log/. /mnt/newlog/
sudo umount /mnt/newlog
sudo rmdir /mnt/newlog

# /etc/fstab
/dev/chakal-vg/varlog /var/log ext4 defaults,nodev,nosuid,noexec 0 2
```

O `cp -a` preserva donos, permissões e ACLs. Ele substituiu o `rsync` do roteiro para evitar instalar mais um pacote. O `/.` no fim do caminho copia também as permissões do próprio diretório.

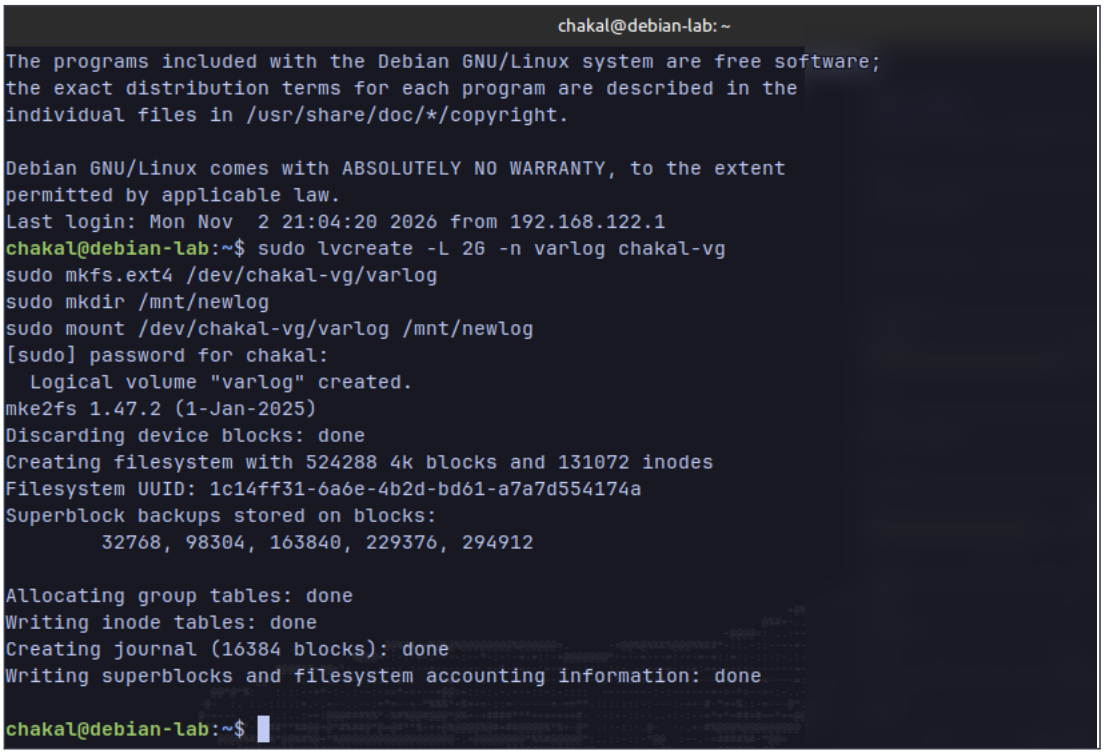


Figura 55. *Volume* `varlog` *de 2 GB criado e formatado.*

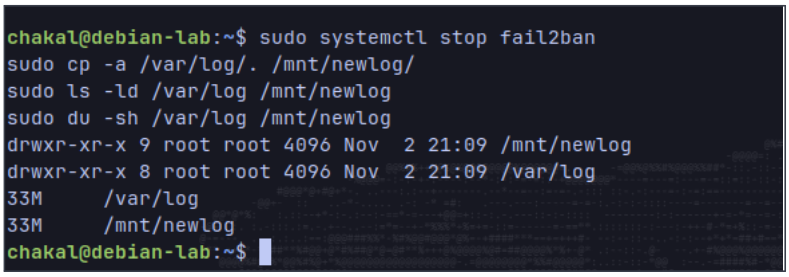


Figura 56. *Os dois lados têm 33 MB e as mesmas permissões. O* `9` *contra* `8` *vem da pasta* `lost+found` *do volume novo.*

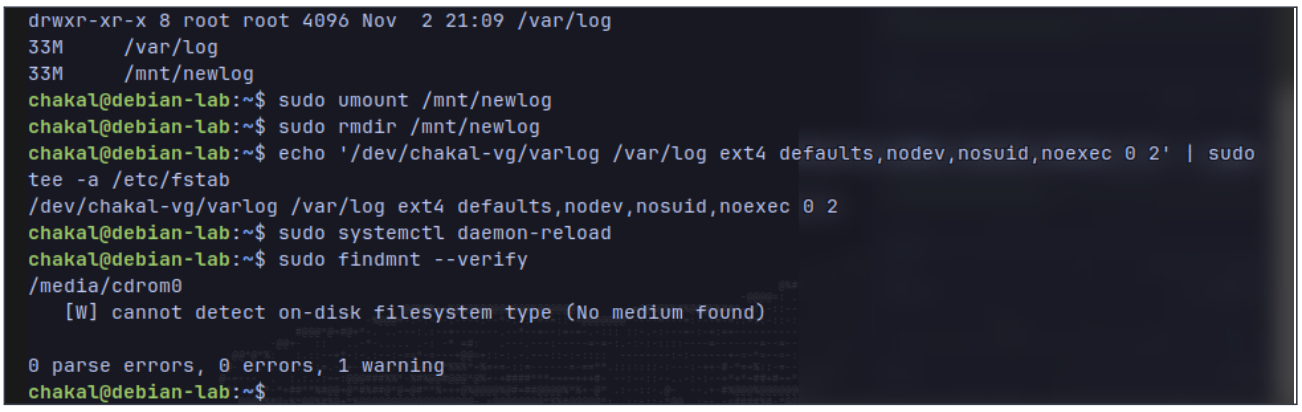


Figura 57. *Linha adicionada ao* `fstab` *e validação com* `0 errors`*.*

Conferência depois do reinício

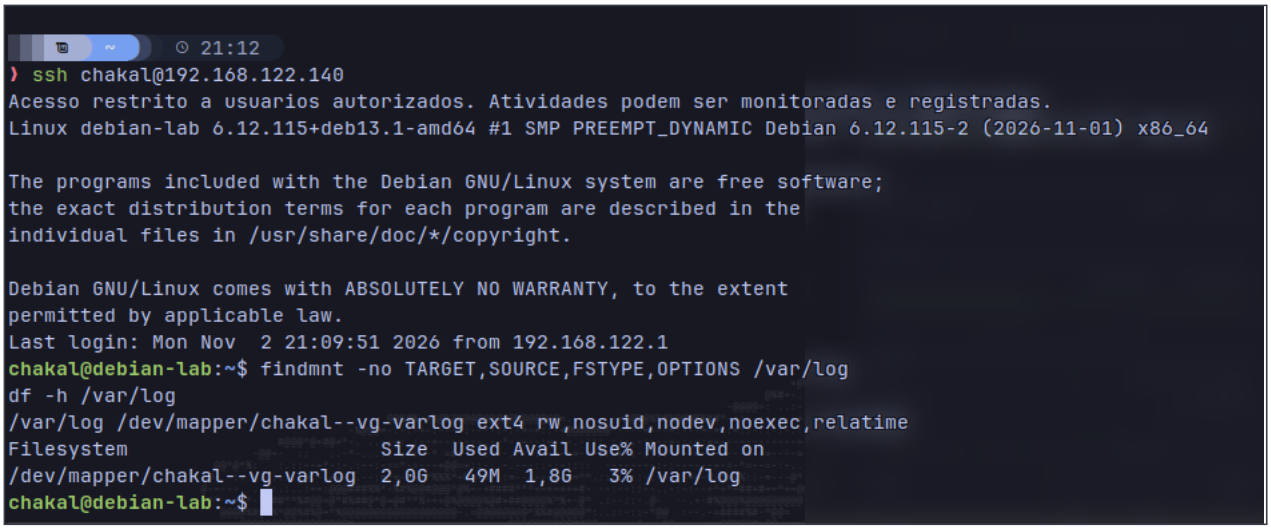


Figura 58. `/var/log` *montado no volume novo, com* `nosuid`, `nodev` *e* `noexec`.

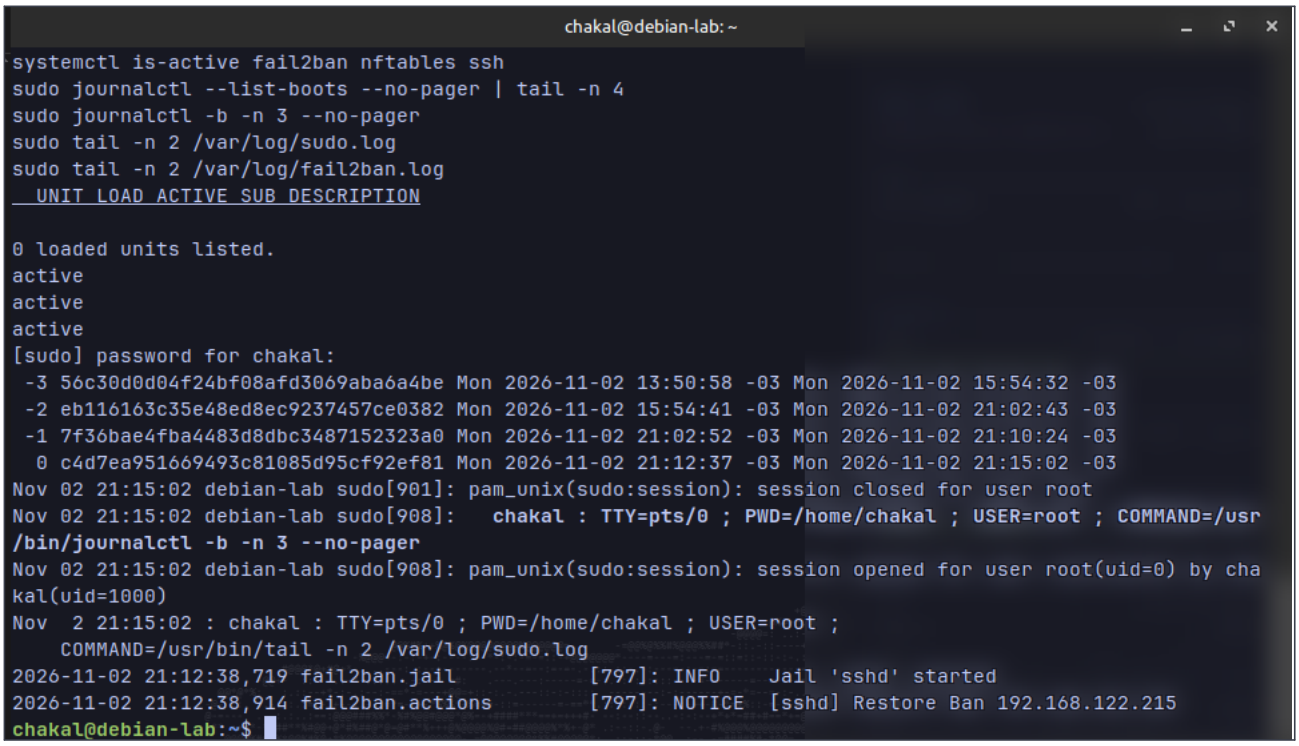


Figura 59. *Nenhuma unidade com falha, serviços ativos, histórico de boots preservado e logs sendo gravados.*

> **Detalhe que ficou**
> Os arquivos antigos de `/var/log` continuam na raiz, escondidos sob o ponto de montagem, ocupando cerca de 33 MB. Removê-los é opcional e exige desmontar o volume ou acessar a raiz por outro caminho.

> **Resumo para o README**
> **1.7 Montagens:** apliquei `noexec` ao `/tmp` (drop-in em `tmp.mount`) e ao `/dev/shm` (`fstab`), confirmando com scripts bloqueados e um `apt` funcional. Movi o `/var/log` para um LV próprio de 2 GB com `nodev,nosuid,noexec`. Como o root está bloqueado, troquei o modo de manutenção por cópia com o sistema no ar seguida de reinício, aceitando perder os logs do intervalo. O `/var/tmp` ficou de fora nesta etapa.

1.8 Parâmetros do kernel

Objetivo

Ajustar o comportamento do kernel com o `sysctl`, usando um arquivo em `/etc/sysctl.d` para os valores persistirem depois de reiniciar.

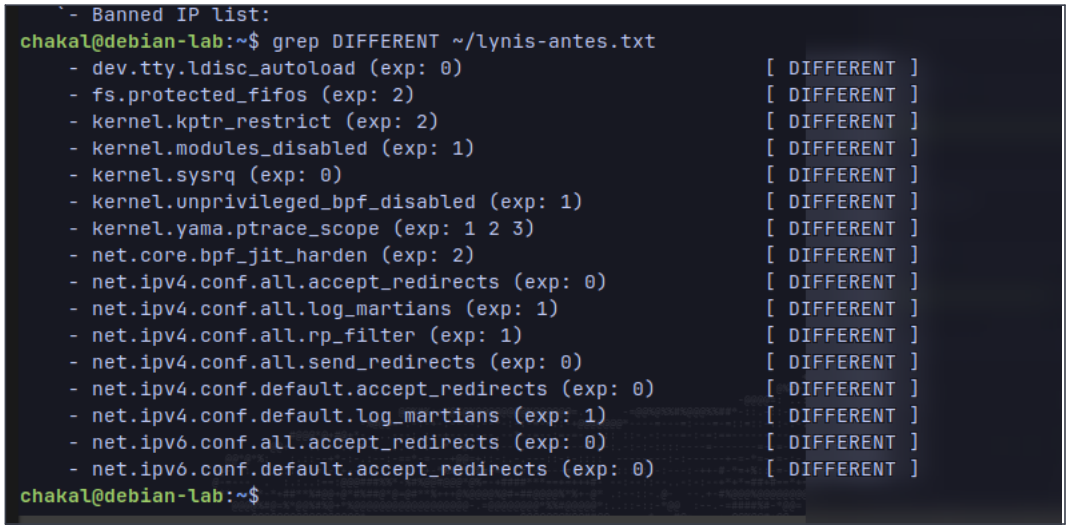


Figura 60. *Os 16 parâmetros marcados pelo Lynis na auditoria inicial.*

Grupo	Parâmetros	Efeito
Kernel	`kptr_restrict`, `dmesg_restrict`, `ptrace_scope`, `sysrq`, `bpf`, `ldisc_autoload`	Esconde informações internas, limita o `ptrace` e desliga recursos pouco usados
Arquivos	`suid_dumpable`, `protected_*`	Impede core dumps de SUID e truques com links e arquivos em diretórios compartilhados
Rede	`rp_filter`, `*_redirects`, `accept_source_route`, `log_martians`, `tcp_syncookies`	Recusa redirecionamentos e roteamento pela origem, descarta origem falsa e protege contra SYN flood

Em relação ao roteiro, incluímos as variantes `default`, que valem para interfaces novas, e parâmetros extras do mesmo tipo. O arquivo completo está no Apêndice A.

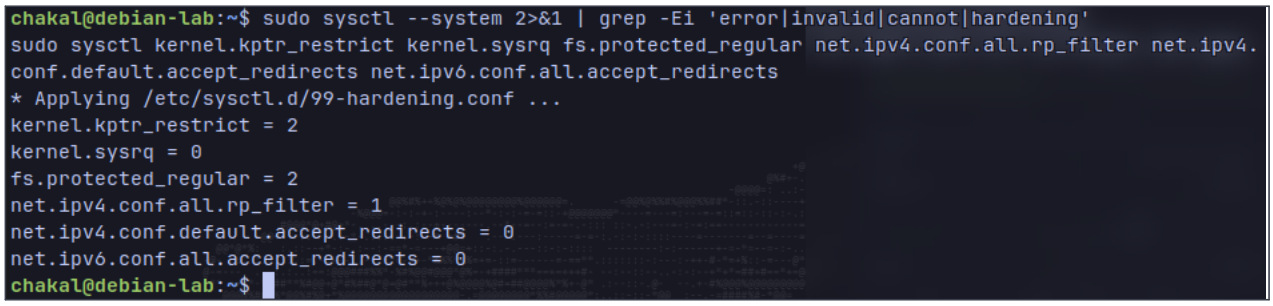


Figura 61. *Arquivo aplicado sem erros e valores conferidos.*

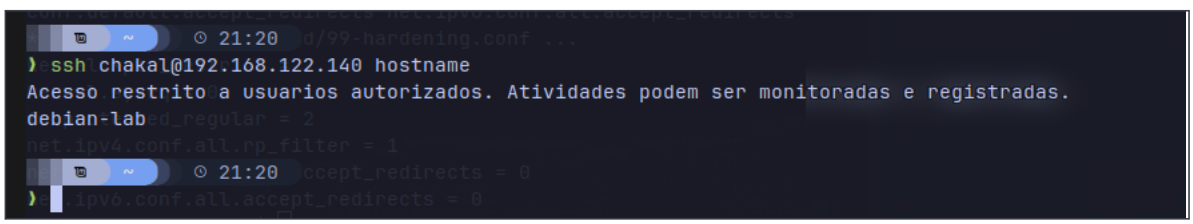


Figura 62. *Um novo login por SSH confirmou que a rede continuou funcionando.*

Dos 16 parâmetros, 15 foram cobertos. O `kernel.modules_disabled = 1` ficou de fora porque proíbe carregar qualquer módulo até o próximo reinício, o que pode quebrar firewall, armazenamento ou

drivers carregados depois.

> **Resumo para o README**
> **1.8 Kernel:** criei `/etc/sysctl.d/99-hardening.conf` com restrições a informações do kernel, `ptrace`, BPF, core dumps de SUID, links e arquivos em diretórios compartilhados, além de proteções de rede em IPv4 e IPv6. Dos 16 parâmetros apontados pelo Lynis, 15 foram cobertos. Mantive `kernel.modules_disabled` desligado para não bloquear o carregamento de módulos.

1.9 Serviços, pacotes e SUID

Objetivo

Reduzir a superfície de ataque removendo o que não tem uso. O que não existe não pode ser explorado.

Pacotes e serviços

```
sudo apt autoremove --purge -y
sudo systemctl disable --now lynis.timer
sudo apt purge -y inetutils-telnet netcat-traditional
```



Figura 63. *Dois pacotes encontrados: o `telnet`, sem criptografia, e o `netcat`, útil para abrir shells reversos ou mover arquivos.*

Os testes da fase 2 partem da Kali e do Ubuntu, então os dois pacotes puderam sair. O `autoremove` removeu sobras do leitor de tela e o kernel `6.12.107`. O `lynis.timer` saiu para não sobrescrever o relatório fora de hora.

```
$ ls /boot/vmlinuz-*
/boot/vmlinuz-6.12.113+deb13-amd64  /boot/vmlinuz-6.12.115+deb13.1-amd64

# Mudanças nos serviços ativados
entraram: fail2ban.service, nftables.service, unattended-upgrades.service
saíram:   lynis.timer, apt-listchanges.timer
```

O `diff` marcou o arquivo inteiro só porque o alinhamento das colunas mudou. A saída do `apt-listchanges.timer` não foi pedida: o pacote continua instalado, e não descobrimos o que desativou o timer. Como o lab não depende dele, deixamos assim.



Figura 64. *O pacote segue instalado, e só o timer está desativado.*

Binários SUID

Binários	Decisão	Motivo
`chfn`, `chsh`, `gpasswd`, `newgrp`, `ssh-keysign`	Remover SUID	O lab não usa essas funções
`passwd`, `su`, `sudo`, `mount`, `umount`, `dbus-daemon-launch-helper`	Manter	Fazem parte de login, administração, montagem ou do D-Bus

Usamos o `dpkg-statoverride` porque ele registra a permissão no dpkg. Assim, uma atualização não devolve o SUID, algo que poderia acontecer com um simples `chmod`.

```
users:(("sshd",pid=805,fd=7))
chakal@debian-lab:~$ for f in /usr/bin/chfn /usr/bin/chsh /usr/bin/gpasswd /usr/bin/newgrp /usr/lib/open
ssh/ssh-keysign; do
  sudo dpkg-statoverride --update --add root root 0755 "$f"
done
sudo find / -xdev -perm -4000 -type f 2>/dev/null | sort | tee ~/suid-depois.txt > /dev/null
diff ~/suid-antes.txt ~/suid-depois.txt
1,3d0
< /usr/bin/chfn
< /usr/bin/chsh
< /usr/bin/gpasswd
5d1
< /usr/bin/newgrp
11d6
< /usr/lib/openssh/ssh-keysign
chakal@debian-lab:~$
```

Figura 65. *O `diff` confirma que saíram exatamente os 5 binários planejados.*

Módulos do kernel

Bloqueamos 11 módulos de armazenamento externo, protocolos de rede apontados pelo Lynis e sistemas de arquivos sem uso. O `install ... /bin/false` faz o carregamento falhar.

```
< /usr/lib/openssh/ssh-keysign
chakal@debian-lab:~$ sudo tee /etc/modprobe.d/hardening.conf > /dev/null <<'EOF'
install usb-storage /bin/false
install firewire-core /bin/false
install dccp /bin/false
install sctp /bin/false
install rds /bin/false
install tipc /bin/false
install cramfs /bin/false
install freevxfs /bin/false
install hfs /bin/false
install hfsplus /bin/false
install jffs2 /bin/false
EOF
lsmod | grep -E '^(usb_storage|firewire_core|dccp|sctp|rds|tipc|cramfs|freevxfs|hfs|hfsplus|jffs2)\b'
sudo modprobe sctp; echo "codigo de saida: $?"
modprobe: ERROR: Error running install command '/bin/false' for module sctp: retcode 1
modprobe: ERROR: could not insert 'sctp': Invalid argument
codigo de saida: 1
chakal@debian-lab:~$
```

Figura 66. *Nenhum desses módulos estava carregado, e o `modprobe sctp` falhou com código 1, como esperado.*

```
netcat-traditional
chakal@debian-lab:~$ sudo whoami
root
chakal@debian-lab:~$ systemctl --failed --no-pager
  UNIT LOAD ACTIVE SUB DESCRIPTION

0 loaded units listed.
chakal@debian-lab:~$
```

Figura 67. *`sudo` funcionando e nenhuma unidade com falha.*

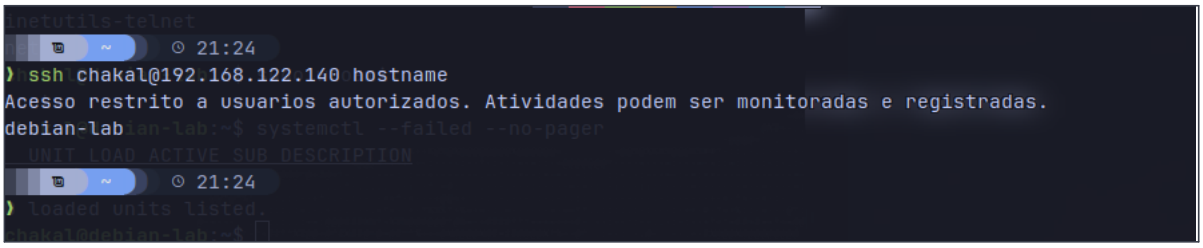


Figura 68. *SSH funcionando a partir de um novo terminal.*

> **Resumo para o README**
> **1.9 Superfície de ataque:** removi o kernel antigo e pacotes sem uso, incluindo `inetutils-telnet` e `netcat-traditional`, e desativei o `lynis.timer`. Tirei o SUID de `chfn`, `chsh`, `gpasswd`, `newgrp` e `ssh-keysign` com `dpkg-statoverride`, reduzindo de 11 para 6 binários. Bloqueei 11 módulos de armazenamento externo, protocolos e sistemas de arquivos sem uso.

1.10 Auditoria com auditd

Objetivo

Registrar eventos no nível do kernel, como alterações em arquivos sensíveis e comandos executados como root. Esses registros vão alimentar os cenários de detecção da fase 3.

```
chakal@debian-lab:~$ sudo tee /etc/audit/rules.d/hardening.rules > /dev/null <<'EOF'
-w /etc/passwd -p wa -k identidade
-w /etc/shadow -p wa -k identidade
-w /etc/group -p wa -k identidade
-w /etc/sudoers -p wa -k sudoers
-w /etc/sudoers.d/ -p wa -k sudoers
-w /etc/ssh/sshd_config -p wa -k sshd
-w /etc/ssh/sshd_config.d/ -p wa -k sshd
-w /etc/nftables.conf -p wa -k firewall
-w /etc/pam.d/ -p wa -k pam
-a always,exit -F arch=b64 -S execve -F euid=0 -F auid>=1000 -F auid!=unset -k root_cmds
-a always,exit -F arch=b32 -S execve -F euid=0 -F auid>=1000 -F auid!=unset -k root_cmds
EOF
chakal@debian-lab:~$
```

Figura 69. *Regras criadas. Em relação ao roteiro, acrescentamos a regra* `b32`*, o firewall e o PAM.*

- `-w` **com** `-p wa`**:** vigia um arquivo ou diretório, registrando escrita e mudança de atributos.
- `-k`**:** define uma etiqueta para buscar o evento depois.
- `-S execve`**:** registra execuções de programas como root feitas por um usuário humano que fez login.
- `auid`**:** guarda quem logou originalmente, mesmo depois do `sudo`.
- `arch=b32`**:** impede que um programa de 32 bits escape do registro.

Resultado

```
$ sudo auditctl -s | grep -E '^(enabled|lost|backlog) '
enabled 1
lost 0
backlog 0

$ sudo auditctl -l
-w /etc/passwd -p wa -k identidade
...
-a always,exit -F arch=b32 -S execve -F euid=0 -F auid>=1000 -F auid!=-1 -F key=root_cmds

$ systemctl is-active auditd
active
```

As 11 regras foram carregadas. O aviso `Old style watch rules are slower` trata só de desempenho e não afeta os registros.

Teste de detecção

```
sudo touch /etc/sudoers.d/teste-audit
sudo rm /etc/sudoers.d/teste-audit
sudo ausearch -k sudoers -i --start recent | tail -n 12

# Trechos do evento
type=PATH    name=/etc/sudoers.d/teste-audit nametype=CREATE
type=SYSCALL syscall=openat   auid=chakal uid=root comm=touch key=sudoers
type=PATH    name=/etc/sudoers.d/teste-audit nametype=DELETE
type=SYSCALL syscall=unlinkat auid=chakal uid=root comm=rm    key=sudoers
```

O ponto principal é `auid=chakal` com `uid=root`: mesmo com `sudo`, o evento mostra quem iniciou a ação. A regra `root_cmds` também funcionou. O evento mostrado na saída foi o próprio `ausearch`, porque ele rodou como root logo depois do `id`.

Resumo para o README
1.10 auditd: criei regras para identidades, sudoers, SSH, firewall e PAM, além do registro de comandos executados como root por usuários logados, em 32 e 64 bits. Em teste, uma alteração em `/etc/sudoers.d` gerou eventos com `auid=chakal`, mantendo a identidade original mesmo depois do `sudo`.

1.11 AppArmor e AIDE

Objetivo

O AppArmor limita o que cada programa pode acessar. O AIDE tira uma foto dos arquivos e aponta o que mudou depois.

AppArmor

```
$ sudo aa-status | grep -E 'profiles are|processes are'
126 profiles are loaded.
6 profiles are in enforce mode.
44 profiles are in complain mode.
0 processes are in enforce mode.
0 processes are unconfined but have a profile defined.

$ sudo aa-unconfined
805 /usr/sbin/sshd (...) not confined
864 /usr/sbin/dhcpcd not confined
```

Limitação importante
O AppArmor está ativo, mas **nenhum processo em execução está confinado**. O `sshd` e o `dhcpcd` rodam sem perfil, então ele ainda não protege esses serviços. O ponto fica para a fase 2.

AIDE

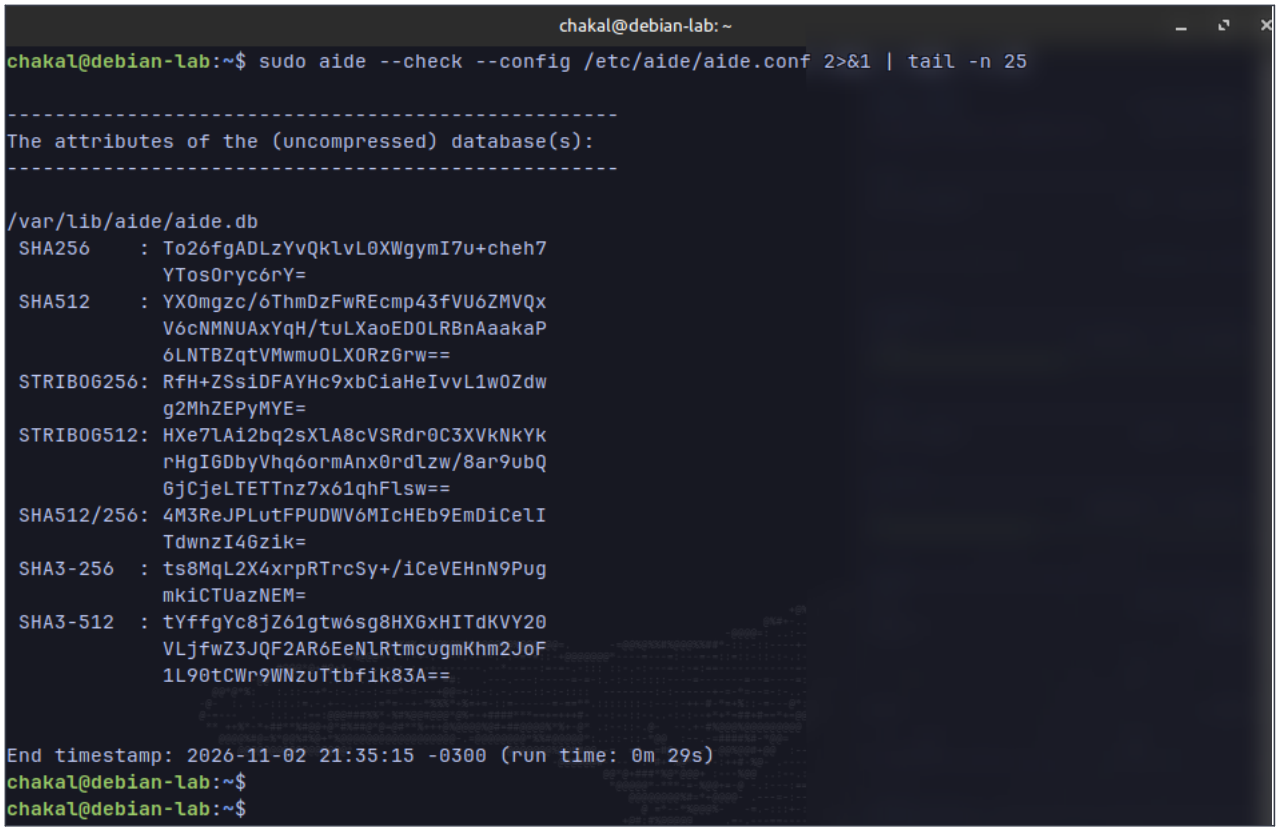


Figura 70. *Primeira verificação. Cada execução levou cerca de 30 segundos, e a base usa SHA256, SHA512 e outros algoritmos.*



Figura 71. *Teste: o AIDE detectou o arquivo `/etc/teste-aide`, mas havia mais uma entrada adicionada.*

O achado: Exim instalado sem querer

Em vez de ignorar a segunda entrada, listamos o relatório completo. O arquivo `/run/exim4/exim.pid` apareceu como novo, e o `aa-unconfined` mostrou um processo que não deveria existir:

```
f++++++++++++++++++: /run/exim4/exim.pid
21991 /usr/sbin/exim4 not confined

$ sudo ss -tulpn | grep -E ':25\b'
tcp LISTEN 0 20 127.0.0.1:25 0.0.0.0:* users:(("exim4",pid=21991,fd=5))
tcp LISTEN 0 20     [::1]:25    [::]:* users:(("exim4",pid=21991,fd=6))

$ sudo find / -xdev -perm -4000 -type f 2>/dev/null | sort | diff ~/suid-depois.txt -
6a7
> /usr/sbin/exim4
```

O `history.log` do APT confirmou que `exim4-*` e `bsd-mailx` entraram como pacotes automáticos no `apt install -y aide`, às 21:33:13. A simulação da remoção não incluiu o `aide`, o que mostra que eles

não eram dependências obrigatórias. O serviço estava preso a `localhost`, mas trazia três problemas:

- adicionava um serviço fora do plano;
- desfazia parte do trabalho do 1.9, com um novo binário SUID;
- entraria em conflito com o Postfix na fase 2.

```
sudo apt purge --simulate exim4-base exim4-config exim4-daemon-light   # confere antes
sudo apt purge -y bsd-mailx exim4-base exim4-config exim4-daemon-light
sudo apt autoremove --purge -y
```

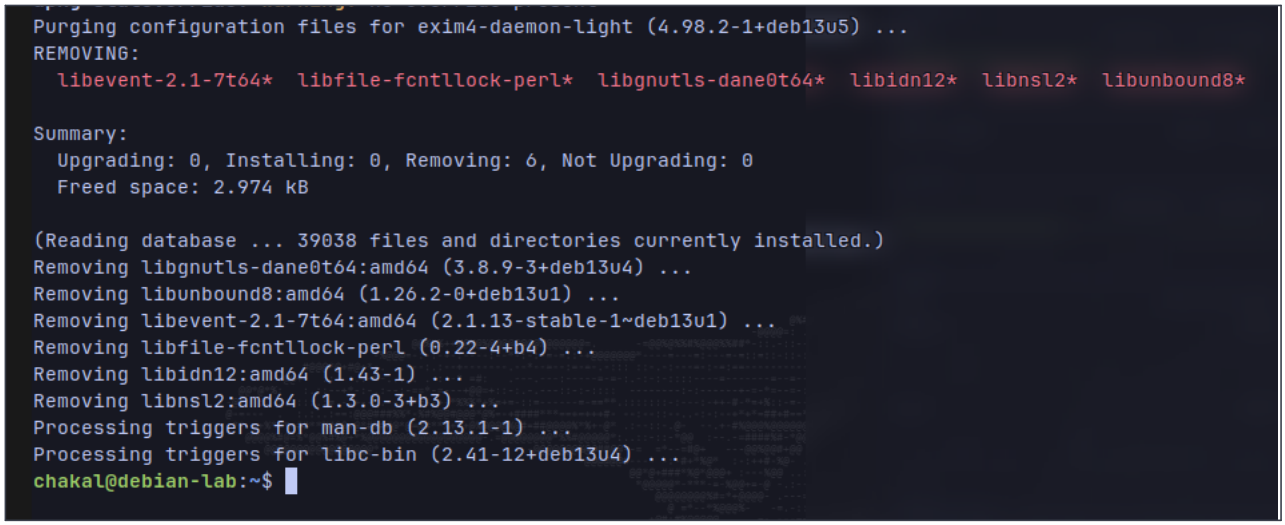


Figura 72. *Remoção das bibliotecas que tinham vindo com o Exim.*



Figura 73. *Porta 25 fechada, SUID de volta aos 6 esperados, nenhuma unidade com falha e* `aide` *ainda instalado.*

Prevenção

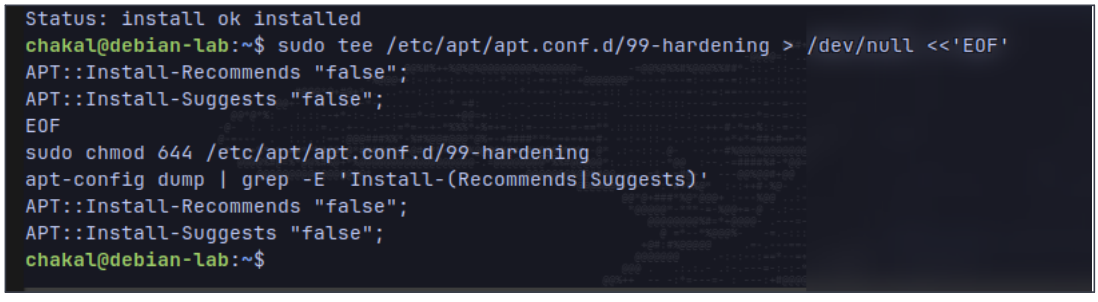


Figura 74. *O* `apt` *deixou de instalar recomendações e sugestões automaticamente.*

Também excluímos do AIDE o que muda o tempo todo, como `/dev`, `/run` e logs em atualização constante. Assim, um alerta real como esse fica mais fácil de ver. Depois, refizemos a base.

```
sudo aide --config-check --config /etc/aide/aide.conf
sudo aideinit -y -f
sudo cp /var/lib/aide/aide.db.new /var/lib/aide/aide.db
Running aide --init...
chakal@debian-lab:~$ sudo aide --check --config /etc/aide/aide.conf 2>&1 | grep -E '^[fdl].*: /'
f =.... mc..H.. .  : /var/lib/dhcpcd/enp1s0.lease
chakal@debian-lab:~$
```

Figura 75. *Com a base refeita, só apareceu a concessão do DHCP, que muda a cada renovação.*

Consequência
Sem servidor de e-mail, os relatórios automáticos do AIDE não são enviados por e-mail até a fase 2. A verificação manual continua funcionando.

Resumo para o README
1.11 AppArmor e AIDE: instalei as ferramentas e os perfis extras do AppArmor. Ele está ativo, mas `sshd` e `dhcpcd` rodam sem perfil, o que fica como limitação registrada. Criei a base do AIDE, excluindo `/dev`, `/run` e logs de alta rotatividade, e testei a detecção de um arquivo novo em `/etc`. O próprio AIDE revelou que o pacote trouxe o Exim como recomendação, abrindo a porta 25 em `localhost` e um novo binário SUID. Removi o Exim e configurei o `apt` para não instalar recomendações automaticamente.

1.12 Banner legal

O aviso do SSH já tinha sido criado no passo 1.4 e aparece nos prints de login. Faltava repetir o texto em `/etc/issue`, mostrado no console local:

```
sudo cp /etc/issue.net /etc/issue
cat /etc/issue
```

Sem evidência
Não há print da saída deste comando. O banner do SSH está comprovado, mas o do console ainda precisa de confirmação com `cat /etc/issue`.

Resumo para o README
1.12 Banner: configurei o aviso legal em `/etc/issue.net` (SSH) e `/etc/issue` (console), sem revelar o sistema operacional.

1.13 Auditoria final

Primeira medição

A nova auditoria, com a mesma versão do Lynis, deu **83**. Os avisos caíram para 1, e as sugestões, para 25. O relatório detalhado mostrou o que ainda restava:



Figura 76. *Permissões de cron e do* `sshd_config` *eram os únicos pontos simples que restavam, além de decisões já tomadas.*

Ajustes finais

```
sudo apt install -y needrestart debsums apt-show-versions

# /etc/needrestart/conf.d/10-hardening.conf
$nrconf{restart} = 'l';

sudo chmod 600 /etc/crontab /etc/ssh/sshd_config
sudo chmod 700 /etc/cron.d /etc/cron.daily /etc/cron.hourly /etc/cron.weekly /etc/cron.monthly
```

- `needrestart`**:** lista serviços que precisam reiniciar depois de atualizações. O modo `l` só lista, sem reiniciar nada, para não derrubar o SSH.
- `debsums`**:** compara os arquivos instalados com os hashes dos pacotes.
- `apt-show-versions`**:** mostra o estado das versões instaladas.

Depois do caso do Exim, conferimos a superfície de ataque também nesta instalação: portas e SUID não mudaram, e o `debsums -s` não acusou arquivos alterados.



Figura 77. `needrestart` *indica o kernel certo em uso (*`KSTA: 1`*), e portas e SUID continuam iguais.*

```
 users:(("sshd",pid=805,fd=7))
chakal@debian-lab:~$ sudo chmod 600 /etc/crontab /etc/ssh/sshd_config
sudo chmod 700 /etc/cron.d /etc/cron.daily /etc/cron.hourly /etc/cron.weekly /etc/cron.monthly
sudo sshd -t && echo "sshd ok"
systemctl is-active cron ssh
sshd ok
active
active
chakal@debian-lab:~$
```

Figura 78. *Permissões restringidas.* `sshd -t` *passou, e* `cron` *e* `ssh` *seguem ativos.*

Medição final

```
chakal@debian-lab:~$ sudo lynis audit system --no-colors | tee ~/lynis-depois.txt > /dev/null
sudo grep hardening_index /var/log/lynis-report.dat
sudo grep -E '^(warning|suggestion)\[\]=' /var/log/lynis-report.dat | tee ~/lynis-achados-depois.txt | c
ut -d'|' -f1 | sort | uniq -c
hardening_index=84
      1 suggestion[]=ACCT-9622
      1 suggestion[]=ACCT-9626
      1 suggestion[]=AUTH-9230
      1 suggestion[]=BANN-7126
      1 suggestion[]=BANN-7130
      1 suggestion[]=BOOT-5122
      1 suggestion[]=BOOT-5180
      1 suggestion[]=BOOT-5264
      1 suggestion[]=DEB-0280
      1 suggestion[]=DEB-0810
      2 suggestion[]=FILE-6310
      1 suggestion[]=FINT-4402
      1 suggestion[]=HRDN-7230
      1 suggestion[]=KRNL-6000
      1 suggestion[]=LOGG-2154
      1 suggestion[]=LYNIS
      1 suggestion[]=NAME-4028
      1 suggestion[]=NETW-2705
      1 suggestion[]=PKGS-7370
      1 suggestion[]=SSH-7408
      1 suggestion[]=TOOL-5002
      1 warning[]=NETW-2705
chakal@debian-lab:~$
```

Figura 79. Hardening index 84*, com 1 aviso e 22 sugestões.*

```
permitted by applicable law.
┌──(kali㉿kali)-[~]
└─$ sudo nmap -sS -p- -T4 192.168.122.140
[sudo] password for kali:
Starting Nmap 7.99 ( https://nmap.org ) at 2026-11-02 19:52 -0500
Nmap scan report for debian-lab (192.168.122.140)
Host is up (0.00059s latency).
Not shown: 65534 filtered tcp ports (no-response)
PORT   STATE SERVICE
22/tcp open  ssh
MAC Address: 52:54:00:5B:7F:D5 (QEMU virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 87.54 seconds
┌──(kali㉿kali)-[~]
└─$
```

Figura 80. *Segunda varredura pela Kali, com o mesmo resultado: só a* `22/tcp` *aberta.*

Fechamento



Figura 81. *Inventários finais e pacote com os 15 arquivos de configuração.*

```
$ scp 'chakal@192.168.122.140:~/*-depois.txt' ~/debian-hardening-lab/evidencias/
$ scp chakal@192.168.122.140:~/configs-fase1.tar.gz ~/debian-hardening-lab/

$ grep -RiE 'password|secret|private key' ~/debian-hardening-lab/configs
# Resultado: só comentários do login.defs e a linha "PasswordAuthentication no"
```



Figura 82. *Base final do AIDE. A verificação não listou nenhuma diferença.*

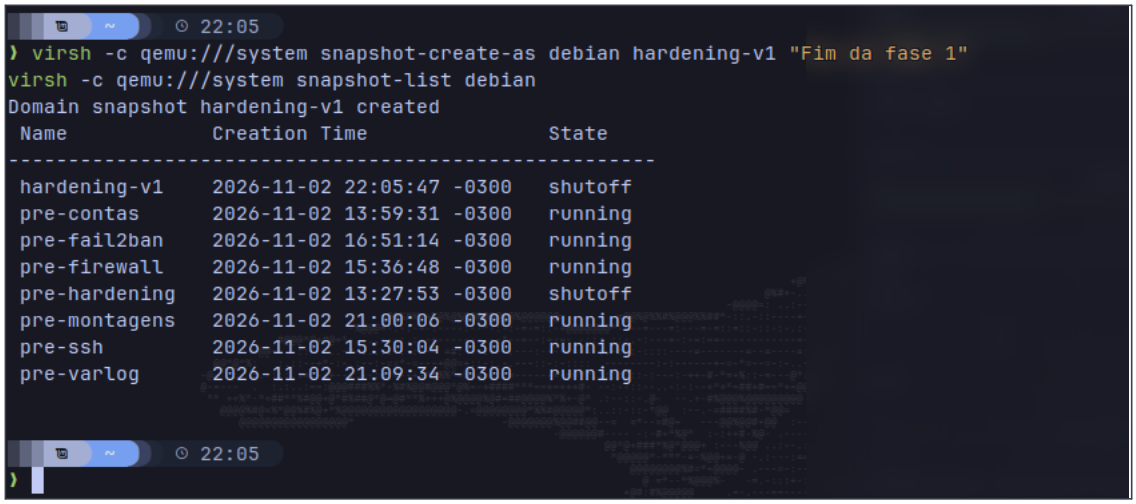


Figura 83. *Snapshot* `hardening-v1` *criado com a VM desligada.*

Resumo para o README
1.13 Resultado: o hardening index subiu de 63 para 84, os avisos caíram de 2 para 1, e as sugestões, de 48 para 22. A varredura pela Kali mostra só o SSH aberto, e os binários SUID caíram de 11 para 6. O aviso restante é esperado na rede NAT do lab, e as sugestões restantes foram justificadas ou adiadas para as fases 2 e 3.

4. Problemas e aprendizados

Os problemas abaixo apareceram durante a execução. Vários deles são erros comuns no dia a dia de administração.

Passo	Sintoma	Causa	Solução ou aprendizado
1.0	`invalid URI qemu://system`	Faltou uma barra	Usar `qemu:///system`
1.2	`tee: invalid option`	Barras `\` juntaram três comandos	Colar um comando por linha
1.2	`scp` pediu senha dentro da VM	Comando rodou no Debian	Olhar o prompt antes de executar
1.2	Pacote não encontrado	Etapa de instalação não rodou	Validar antes de seguir
1.3	`bad permissions` no sudoers	Arquivo não estava em `0440`	Usar `chmod 0440` e `visudo -c`
1.5	`ssh` para a Kali recusado	Serviço desligado e usuário errado	Ativar o SSH e informar o usuário
1.5	Comandos não rodaram na VM	Colados junto com o `ssh`	Esperar o prompt remoto
1.5	Símbolos errados no console	Teclado físico US, console ABNT2	Trocar o layout para `us`
1.6	`jail.local` com 1 byte	Conteúdo não estava salvo	Conferir o que o serviço carregou, não só a sintaxe
1.6	`Failed to access socket`	`status` rodou durante a subida	Esperar alguns segundos
1.6	Kali não estava bloqueada	Ban de 1 hora já expirado	Conferir o log antes de concluir
1.6	`Restore Ban` após reiniciar	Tentativa de teste gerou novo ban	Lembrar que todo teste conta para o limite
1.11	Porta 25 e SUID novos	Exim veio como recomendação	Conferir portas e SUID depois de cada instalação

5. Desvios do roteiro

Estas mudanças devem entrar na próxima versão do `roteiro-projeto-debian-lab.pdf`.

Passo	Roteiro v1.0	O que foi feito	Motivo
Seção 2	Teclado ABNT2 e snapshot `limpa`	Console em US; ponto mais antigo é o `pre-hardening`	Refletir o ambiente real
1.1	`ss` sem `sudo`, com cores	`sudo`, `--no-colors` e `tee`	Ver processos e gerar evidências limpas
1.3	Editar `pwquality.conf`	Drop-in em `pwquality.conf.d`	Seguir a regra de drop-ins
1.4	12 diretivas	16 diretivas e banner antecipado	Atender sugestões do Lynis
1.5	Sem regra de DHCP	Porta 68 liberada a partir do gateway	Manter a renovação do IP

Passo	Roteiro v1.0	O que foi feito	Motivo
1.6	Sem `ignoreip`	Mint no `ignoreip`	Evitar banir o host administrativo
1.6	Sem aviso sobre o nftables	Reiniciar o fail2ban depois de recarregar as regras	O `flush ruleset` apaga as regras dele
1.7	`findmnt` com 3 caminhos	`findmnt -T` em um laço	O comando não aceita vários caminhos juntos
1.7	`rescue.target` e `rsync`	Cópia com `cp -a` e reinício	Root bloqueado e menos pacotes
1.8	Só variantes `all`	`all`, `default` e parâmetros extras	Cobrir interfaces novas
1.10	Sem `b32`, firewall e PAM	Três regras a mais	Ampliar a cobertura
1.11	`apt install -y aide`	Usar `--no-install-recommends`	Evitar o Exim

6. Decisões sobre as sugestões restantes

Todas as 22 sugestões e o aviso restante têm uma decisão. A lista completa está em `lynis-achados-depois.txt`.

Item	Decisão	Justificativa
`NETW-2705` (aviso e sugestão)	Risco aceito	A VM usa só o DNS da rede NAT do lab
`LOGG-2154`, `ACCT-9622`, `ACCT-9626`, `HRDN-7230`	Fase 3	Logs externos, métricas e detecção entram com o monitoramento
`NAME-4028`	Fase 2	O domínio `lab.internal` será configurado no e-mail
`BOOT-5264`	Fase 2	O sandbox de serviços faz mais sentido com o e-mail no ar
`BOOT-5122`	Risco aceito	Quem acessa o host controla a VM de qualquer forma
`FILE-6310` (2 itens)	Risco aceito	`/var/log` já está separado, e `/home` não justifica outro volume no lab
`SSH-7408`	Risco aceito	Trocar a porta 22 só despista varreduras simples
`AUTH-9230`	Risco aceito	O `YESCRYPT` já é forte
`DEB-0280`, `DEB-0810`	Risco aceito	Mexem no PAM ou no fluxo do `apt`, com pouco ganho no lab
`KRNL-6000`	Risco aceito	Resta só o `kernel.modules_disabled`
`BANN-7126`, `BANN-7130`	Provável falso positivo	O Lynis parece procurar palavras em inglês, e o banner está em português
`FINT-4402`	Provável falso positivo	A base do AIDE já usa SHA256 e SHA512
`PKGS-7370`	Aplicado	`debsums` instalado e testado, embora a sugestão continue aparecendo
`LYNIS`, `BOOT-5180`, `TOOL-5002`	Sem ação	Itens informativos, e a versão do Lynis foi mantida de propósito

Outro item do relatório, o `hidepid` em `/proc`, também ficou como risco aceito: ele pode atrapalhar serviços do systemd, e a VM tem um único usuário humano.

7. Limitações e pontos sem comprovação

Para manter o relatório fiel às evidências, estes pontos ficam registrados:

- `/etc/issue`**:** não há print que confirme a cópia do banner para o console.
- **Fuso da Kali:** a última varredura ainda mostrou `-0500`, e não há print da correção. Isso precisa estar certo antes da fase 3.
- **Testes de rede:** as varreduras cobriram só TCP sobre IPv4.
- **AppArmor:** nenhum processo em execução está confinado.
- **Política de senha:** ela só vale na próxima troca. Não foi verificado se a senha atual do `chakal` já atende aos 14 caracteres.
- `/var/tmp`**:** ficou sem `noexec`, por ser a saída comum para o `apt` quando o `/tmp` tem essa opção.
- `noexec`**:** não impede que um interpretador execute um script.
- **Reinício do kernel:** atualizações automáticas do kernel dependem de um reinício manual.
- **Nota do Lynis:** o índice mede a aderência a verificações, não a segurança real do sistema.

8. Próximos passos

Antes da fase 2

- Confirmar `/etc/issue` e o fuso da Kali.
- Atualizar o roteiro com os desvios da seção 5.
- Remover `configs-fase1.tar.gz` do repositório, já que o conteúdo está extraído em `configs/`.
- Rever `etc/fstab` e `etc/login.defs` antes de publicar.
- Escrever `docs/fase1-hardening.md` e publicar `configs/` e `evidencias/`.

Durante a fase 2

- Instalar pacotes conferindo permissões, porque o `UMASK` e o `apt` estão mais restritivos.
- Instalar de forma explícita as recomendações necessárias.
- Repetir a checagem de portas e SUID depois de cada instalação.
- Rever AppArmor e sandbox de serviços com Postfix e Dovecot.
- Refazer a base do AIDE ao final.

Durante a fase 3

- Cobrir logs externos, métricas e scanner de malware.
- Usar `auditd`, `nft-drop` e fail2ban como fontes dos cenários de detecção.
- Incluir varreduras UDP e IPv6.

Apêndice A. Configurações finais

Resumo dos arquivos alterados. O pacote `configs-fase1.tar.gz` guarda 15 arquivos. Ficaram fora dele `/etc/issue`, `/etc/default/keyboard` e as permissões de cron e do `sshd_config`.

Arquivo	Passo
`/etc/security/pwquality.conf.d/hardening.conf`	1.3
`/etc/sudoers.d/hardening`	1.3
`/etc/login.defs`	1.3
`/etc/ssh/sshd_config.d/10-hardening.conf`	1.4
`/etc/issue.net` e `/etc/issue`	1.4 e 1.12
`/etc/nftables.conf`	1.5
`/etc/fail2ban/jail.local`	1.6
`/etc/systemd/system/tmp.mount.d/hardening.conf`	1.7
`/etc/fstab`	1.7
`/etc/sysctl.d/99-hardening.conf`	1.8
`/etc/modprobe.d/hardening.conf`	1.9
`/etc/audit/rules.d/hardening.rules`	1.10
`/etc/aide/aide.conf.d/99_hardening`	1.11
`/etc/apt/apt.conf.d/99-hardening`	1.11
`/etc/needrestart/conf.d/10-hardening.conf`	1.13

/etc/sysctl.d/99-hardening.conf

```
# Kernel
kernel.kptr_restrict = 2
kernel.dmesg_restrict = 1
kernel.yama.ptrace_scope = 1
kernel.sysrq = 0
kernel.core_uses_pid = 1
kernel.unprivileged_bpf_disabled = 1
net.core.bpf_jit_harden = 2
dev.tty.ldisc_autoload = 0

# Sistema de arquivos
fs.suid_dumpable = 0
fs.protected_hardlinks = 1
fs.protected_symlinks = 1
fs.protected_fifos = 2
fs.protected_regular = 2

# Rede IPv4
net.ipv4.conf.all.rp_filter = 1
net.ipv4.conf.default.rp_filter = 1
net.ipv4.conf.all.accept_redirects = 0
net.ipv4.conf.default.accept_redirects = 0
net.ipv4.conf.all.secure_redirects = 0
net.ipv4.conf.default.secure_redirects = 0
net.ipv4.conf.all.send_redirects = 0
net.ipv4.conf.default.send_redirects = 0
net.ipv4.conf.all.accept_source_route = 0
net.ipv4.conf.default.accept_source_route = 0
net.ipv4.conf.all.log_martians = 1
net.ipv4.conf.default.log_martians = 1
```

```
net.ipv4.icmp_echo_ignore_broadcasts = 1
net.ipv4.tcp_syncookies = 1

# Rede IPv6
net.ipv6.conf.all.accept_redirects = 0
net.ipv6.conf.default.accept_redirects = 0
net.ipv6.conf.all.accept_source_route = 0
net.ipv6.conf.default.accept_source_route = 0
```

/etc/modprobe.d/hardening.conf

```
install usb-storage /bin/false
install firewire-core /bin/false
install dccp /bin/false
install sctp /bin/false
install rds /bin/false
install tipc /bin/false
install cramfs /bin/false
install freevxfs /bin/false
install hfs /bin/false
install hfsplus /bin/false
install jffs2 /bin/false
```

/etc/audit/rules.d/hardening.rules

```
-w /etc/passwd -p wa -k identidade
-w /etc/shadow -p wa -k identidade
-w /etc/group -p wa -k identidade
-w /etc/sudoers -p wa -k sudoers
-w /etc/sudoers.d/ -p wa -k sudoers
-w /etc/ssh/sshd_config -p wa -k sshd
-w /etc/ssh/sshd_config.d/ -p wa -k sshd
-w /etc/nftables.conf -p wa -k firewall
-w /etc/pam.d/ -p wa -k pam
-a always,exit -F arch=b64 -S execve -F euid=0 -F auid>=1000 -F auid!=unset -k root_cmds
-a always,exit -F arch=b32 -S execve -F euid=0 -F auid>=1000 -F auid!=unset -k root_cmds
```

/etc/aide/aide.conf.d/99_hardening e /etc/apt/apt.conf.d/99-hardening

```
!/dev($|/)
!/run($|/)
!/var/log/aide($|/)
!/var/log/audit($|/)
!/var/log/sudo\.log$

APT::Install-Recommends "false";
APT::Install-Suggests "false";
```